AI Infra M0 前置基础学习笔记：从 Python/数学到 Transformer、GPU 与集合通信

数据来源：本笔记的骨架、术语体系与绝大部分判断，取自 AIInfraGuide「模块一：前置知识」的 20 篇公开文章（学习路线总览 + 第 1～6 章），访问日期 **2026-11**。

补充来源（用于交叉核对数字与补齐原文未展开的部分，文中标注「补充」）：NVIDIA CUDA C++ Programming Guide、NVIDIA A100 / H100 / Blackwell 架构白皮书、NVIDIA NCCL 官方文档与 nccl-tests、Vaswani et al. *Attention Is All You Need*（2017）、PyTorch 官方文档（autograd / AMP / torch.profiler）。

标注约定：`【补充】` = AIInfraGuide 原文没有、由上述权威来源补齐的内容；`【推断】` = 基于材料做的推理性结论，未经原文或文档直接陈述；`【量级】` = 只保证数量级正确，不对具体尾数负责。

版权声明：AIInfraGuide 原文版权归原作者所有，本笔记为个人学习用途的二次整理、重组与批注，不替代原文；引用原文观点处已尽量保留其表述与限定条件。NVIDIA 白皮书、CUDA/NCCL 文档、PyTorch 文档、arXiv 论文版权归各自权利人所有。若本文与原文冲突，以原文与官方文档为准。

1.1 一句话定义与边界
1.2 三层架构（外加一个第零层）
1.3 不可能三角：计算、通信、显存
1.4 每个技术牺牲什么、换什么
1.5 用三个数字建立"手感"
2.1 Python 名字绑定：一切"诡异 Bug"的源头
2.1.1 默认可变参数：函数定义时求值
2.1.2 浅拷贝与深拷贝
2.3 GIL 到底限制了什么
2.4 并发模型选择表
2.5 基准测试与 profiling：从"慢"定位到具体代码
2.6 Python 性能优化的优先级
2.7 C/C++：值、指针、引用与生命周期
2.7.1 RAII：把资源绑定到对象生命周期
2.8 编译与链接：一条必须背下来的流水线
2.8.1 三类错误不要混在一起
2.8.2 静态库、动态库与 ABI
2.10 Linux 工具链：AI Infra 的日常工作面
2.10.1 文件、文本流与管道
2.10.2 引号、变量与退出码
2.10.3 进程、信号与远程会话
2.10.4 资源排查命令表
2.10.6 GPU 软件栈：四个容易混淆的部分
3.1 读公式先问的三个问题
3.2 形状直觉：(B,S,H) × (H,V) 到底发生了什么
3.2.1 从 (B,S,H) 到 (B,S,V)
3.2.2 同一套口诀推 6 个常见算子
3.2.3 shape 相同不等于内存布局相同
3.3 Softmax 的概率解释
3.3.1 logits 不是概率
3.3.2 平移不变性 = 数值稳定的依据
3.3.3 LogSumExp 与融合
3.3.4 温度参数
3.4 交叉熵：为什么它就是负对数似然
3.4.1 定义与 one-hot 化简
3.4.2 熵、KL 与交叉熵的关系
3.4.3 Perplexity
3.5 链式法则与反向传播
3.5.1 链式法则
3.5.2 计算图：为什么梯度要"累加"
3.5.3 线性层的反向：三行 shape 检查

3.5.4 Softmax + 交叉熵：p − y
3.6 混合精度的数值问题
3.6.1 浮点数是有限集合
3.6.2 精度与动态范围是两件事
3.6.3 舍入、非结合性、归约顺序
3.6.4 溢出、下溢、NaN 与灾难性消减
3.6.6 混合精度训练的基本模式
3.6.7 Loss Scaling 为什么有用
3.7 综合算例：把数学变成工程判断
3.7.1 为什么除以 √D_h
3.7.2 Online Softmax 的分块合并
4.1 AI Infra 四类日常工作，全部指向 Transformer
4.2 RNN 的两个致命弱点与 Transformer 的两次范式转换
4.3 三条分岔路：为什么 Decoder-only 赢了
4.3.1 Decoder-only 胜出的四条理由
4.4 三种注意力机制的分工
4.5 端到端数据流（一页纸版本）
5.1 历史脉络：Attention 是怎么长成现在这样的
5.2 从信息检索理解 Q、K、V
5.3 五步计算：每一步的 shape 都要能背
第 1 步：线性投影
第 2 步：计算原始注意力分数
第 3 步：缩放
第 4 步：Softmax 归一化
第 5 步：加权求和 + 输出投影
完整公式
5.4 手写实现：单头与多头
5.5 Causal Mask：并行训练时如何"假装"是串行的
5.5.1 为什么需要
5.5.2 掩码矩阵长什么样
5.5.3 两种实现
5.5.4 工程意义：跳过近一半计算
5.6 O(N²) 到底来自哪里：逐项拆 FLOPs
5.7 计算瓶颈：Compute Bound 还是 Memory Bound
5.8 O(N²) 的显存瓶颈：一个让人记住的数字
5.9 FlashAttention 的核心思想（只讲与本章相关的部分）
5.9.1 tiling：不物化 N×N
5.9.2 online softmax 让分块在数学上成立
5.9.3 省下的是什么：HBM 访问量
6.1 MHA：把 d_model 切成 N_h 个子空间
6.2 关键洞察：参数量 vs KV Cache 是两笔账
6.3 三种方案的机制差异
6.4 MLA：换一个方向压缩
7.1 FFN 的角色：token 内的运算
7.2 "展开-压缩"结构与四条理由
7.3 激活函数演进：ReLU → GELU → Swish → SwiGLU
7.4 SwiGLU：为什么是 8/3 倍
7.5 参数量分布：FFN 占 2/3
7.6 张量并行切分 FFN：Megatron 的列切 + 行切
7.6.1 切分方案
7.6.2 为什么列切分能"免费"施加激活
7.6.3 通信开销：每 Block 两次 AllReduce
7.6.4 工程约束：d_ff 必须能被 TP 整除
7.7 MoE：把 FFN 拆成多个专家
7.7.1 核心思想
7.7.2 对 Infra 的三个新问题
8.1 为什么必须注入位置：Attention 是排列等变的
8.2 绝对 vs 相对：两种设计思路
8.3 Sinusoidal 编码
8.3.1 公式
8.3.2 收音机调谐的直觉
8.3.3 实现
8.4 可学习位置编码
8.5 RoPE：旋转位置编码
8.5.1 核心思想
8.5.2 为什么旋转能编码相对位置
8.5.3 实现（来自 Decoder Block 章节的完整代码）
8.5.4 三条工程约束（原文重点）
9.1 为什么深层网络难训练
9.2 残差连接：梯度的高速公路
9.3 归一化方法全景

9.4 Pre-Norm vs Post-Norm
9.4.1 梯度流差异（本章最重要的一段）
9.4.2 实验证据与工程结论
10.1 一个 Pre-Norm Decoder-only Block 的数据流
10.2 完整维度表：从输入到 logits
10.3 主流模型维度配置对比
10.4 参数量手算：LLaMA-2-7B 全流程
10.4.1 公式先立好
10.4.2 逐项代入（d=4096, h=32, d_k=128, d_ff=11008, L=32, V=32000）
10.4.3 校验：13B 也要能算
10.5 FLOPs 估算：2N、6N 与 Attention 修正项
10.5.1 三条公式
10.5.2 完整训练算例（这段值得背）
10.5.3 推理的"反直觉"算例
10.6 显存规划：三个公式打天下
10.6.1 权重显存
10.6.2 训练态四大组成
10.6.3 推理态：权重 + KV Cache
10.6.4 完整规划案例：13B 单卡能装多少 batch
11.1 语言模型的本质：条件概率连乘
11.2 从 hidden state 到概率：LM Head + Softmax
11.3 朴素生成循环与它的致命效率问题
11.4 KV Cache：从 $O(S^2)$ 到 $O(S)$
11.4.1 机制
11.4.2 显存代价（这是"显存刺客"）
11.5 Prefill vs Decode：两个阶段的物理本质不同
11.7 采样策略
12.1 两道变换
12.2 三种切分粒度
12.3 BPE：数据驱动的合并
12.4 独热编码为什么失败
12.5 Embedding：一张可学习的查找表
12.6 完整输入流程与 Infra 视角
13.1 Tensor 的四个必会面向
13.2 autograd：三句话记住
13.3 nn.Module 与 state_dict
13.4 标准训练循环（背下来）
13.5 显存与性能分析
14.1 CPU vs GPU：延迟导向 vs 吞吐导向
14.2 层级结构：GPC → TPC → SM → Core
14.3 Warp：GPU 执行的最小单位
14.4 存储层次与 Memory Wall
14.4.1 Roofline 与算术强度
14.5 Tensor Core 与精度格式
14.6 主流 AI GPU 规格对比
14.7 Volta → Blackwell 五代演进
14.8 多卡互联：NVLink / NVSwitch / PCIe / IB
14.8.1 典型拓扑：DGX H100（8 卡）
14.8.2 读 `nvidia-smi topo -m`
14.9 显存管理策略速查
15.1 为什么通信决定并行策略的形状
15.2 单机：NVLink 与 NVSwitch
15.3 多机：InfiniBand 与 RoCE
15.4 五大集合通信原语与通信量公式
15.4.1 数据流示意（4 卡，数据 [a,b,c,d]）
15.5 Ring AllReduce：带宽最优
15.5.1 两个阶段
15.5.2 通信量公式（必背）
15.6 Tree AllReduce：延迟最优
15.7 通信与计算 Overlap
15.8 NCCL：自动挡 + 可观测性
15.8.1 关键环境变量
15.8.2 用 nccl-tests 测真实带宽
15.9 从通信视角解释并行策略
15.9.1 TP 为什么只能待在机内
15.9.2 DP 的通信开销
15.9.3 PP 的气泡
15.9.4 并行策略 ↔ 通信原语映射
15.10 通信瓶颈排查清单
16.1 七条原始标准与模块归属
16.2 模块一：编程语言基础
16.3 模块二：数学基础
16.4 模块三：Transformer（权重最高的模块）
16.5 模块四：PyTorch
16.6 模块五：GPU 硬件

16.7 模块六：集合通信
18.1 一页纸复习清单
18.2 自测题（12 题，答案在后）
18.3 答案
19.1 AIInfraGuide 模块一（一手材料，访问日期 2026–11）
19.2 补充权威来源（用于交叉核对）

第 1 章 AI Infra 全景：三层架构与「计算、通信、显存」不可能三角

1.1 一句话定义与边界

AIInfraGuide 给 AI Infra 的定义是：**用系统工程的方法，把硬件算力高效地"喂"给模型**。原文用了一个很好用的类比：算法研究员是菜单设计者，AI Infra 工程师是搭建厨房、调度厨师、管理食材供应链的后勤总指挥。

这个定义划出了一条清晰的职责边界：

位置	关注对象	典型问题
上层：算法与模型层	模型结构、训练目标、数据配方	用什么架构、loss 怎么设计、数据怎么配比
中间：AI Infra	**算子的执行效率、显存占用、通信开销、服务吞吐**	**这块卡为什么只跑到 30% 利用率**
下层：硬件层	SM、HBM、NVLink、IB 交换机	硬件理论峰值是多少、拓扑长什么样

AI Infra 的使命是**上下两层的翻译官**：把上层模型的计算需求，翻译成下层硬件能高效执行的指令。而当今这个"上层模型"，几乎就等于 Transformer——这是第 4～12 章要花 9 章篇幅讲它的根本原因。

1.2 三层架构（外加一个第零层）

AIInfraGuide 学习路线把整个领域自底向上分为三个核心层级，再加一个前置知识层：

层级	名称	核心关注点	本笔记对应章节
第零层	前置知识	编程语言、数学基础、Transformer 架构、PyTorch、GPU 硬件、通信拓扑	第 2～15 章
第一层	CUDA 编程与算子优化	GPU 架构、存储层次、Kernel 编写、FlashAttention、AI 编译器	仅在第 14 章做硬件铺垫
第二层	分布式训练	数据并行、3D 并行、ZeRO、混合精度	第 15 章做通信铺垫
第三层	推理与部署	KV Cache、PagedAttention、量化、Speculative Decoding	第 11 章做机制铺垫

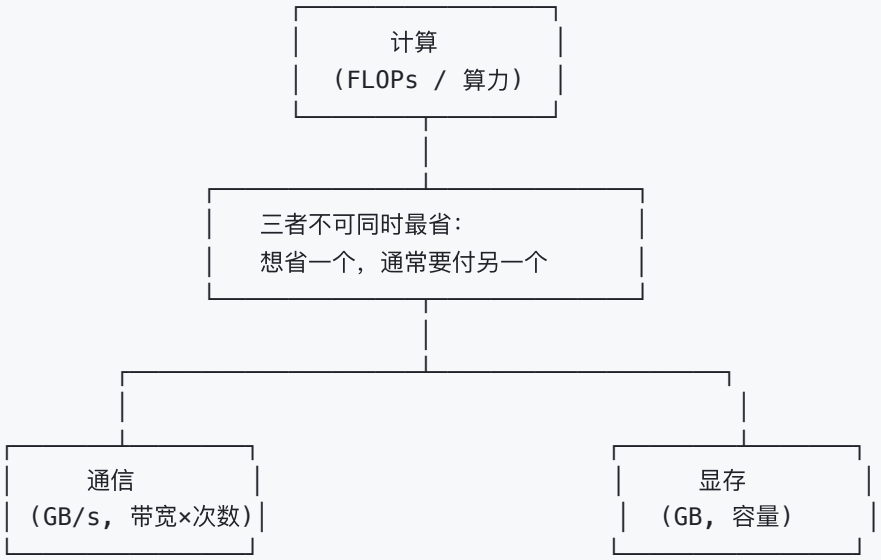
注意每一层的**产物**不同，这决定了你用什么指标衡量它：

层级	你交付的东西	一句话验收指标
第零层	判断力	看到一个模型配置，能算出它要多少显存、瓶颈在哪
第一层	一个 kernel	相对硬件峰值 / 相对 cuBLAS 的达成率
第二层	一个训练任务	MFU、扩展效率、每 step 时间
第三层	一个在线服务	TTFT、TPOT、tokens/s、P95 尾延迟

1.3 不可能三角：计算、通信、显存

原文的核心方法论只有一句话，但它是整条路线的"母题"：

> 所有的优化都是在「计算、通信、显存」这个不可能三角中做取舍。



为什么是"三角"而不是"二选一"？因为这三者在大模型里是**同一份物理资源的不同侧面**：

- **计算**受限于 SM 数量 × 每周期运算量 × 频率（第 14 章）；
- **显存**受限于 HBM 容量，而**显存带宽**决定了计算单元"等数据"的时间（第 14 章 Memory Wall）；
- **通信**受限于 NVLink/IB 带宽 × 通信次数，而通信次数由并行策略的切分方式决定（第 15 章）。

学每一个技术时，原文建议固定问一句：**这个技术牺牲了什么，换取了什么？**

1.4 每个技术牺牲什么、换什么

这张表是全书的"价值换算表"，建议背下来。表中每一项都能在后面章节找到机制解释。

技术	牺牲	换取	机制一句话
ZeRO-1/2/3	通信量（ZeRO-3 约为 DDP 的 1.5 倍）	每卡模型状态显存线性下降到 1/N	把优化器状态/梯度/参数切碎摊到各卡
Activation Checkpointing（重计算）	约 33% 额外计算	激活显存降到约 1/3～1/5	前向只存少量激活，反向重算
混合精度（BF16/FP16）	数值精度、动态范围	约减半的权重显存 + Tensor Core 高算力	低精度算、FP32 累加与 master weights
FP8 训练	精度（需逐层动态缩放）	相对 BF16 约 2 倍理论算力	Transformer Engine 按张量分布切精度
量化（INT8/INT4）	精度	权重显存与带宽成倍下降	用更少比特表示权重/激活
GQA（MHA→GQA）	少量模型质量、KV 头表达力	KV Cache 降到 1/4～1/8	多个 Q 头共享一组 KV 头
MLA	实现复杂度（低秩上下投影）	KV Cache 大幅压缩	只缓存低维 latent，用时解压
张量并行（TP）	通信量（每层 2 次 AllReduce）	单卡放不下的矩阵被切开	按列切 + 按行切，输出处归约
流水线并行（PP）	流水线气泡（GPU 空转）	跨机切层，通信量小	按层切分，micro-batch 流水
数据并行（DP/DDP）	每步一次全量梯度 AllReduce	线性扩展吞吐	各卡算不同数据，梯度求平均
FlashAttention	重算（反向重算注意力）	HBM 访问量与显存从 $O(N^2)$ 物化降下来	tiling + online softmax，不物化 N×N
Continuous Batching	调度复杂度	GPU 利用率从约 30% 拉到 80%+	请求随到随拼，完成即退出
Prefix Cache	显存（缓存 KV 块）	跳过重复前缀的 Prefill	前缀 hash 命中即复用 KV
Speculative Decoding	额外算力（小模型猜 + 大模型验）	降低每 token 的等效延迟	一次前向验证多个候选 token
Chunked Prefill	单请求 TTFT 不变差也不变好	系统吞吐提升、Decode 抖动变小	长 prompt 分块，与 Decode 交错

【推断】这张表有一个隐含的元规律：**显存是最"硬"的约束**（放不下就是放不下，直接 OOM），**通信是最"软"的约束**（可以用 overlap 部分藏起来），**计算是最"便宜"的约束**（多花 33% 计算通常可以接受）。所以业界的优化顺序通常是：先用通信换显存（ZeRO）→ 再用计算换显存（重计算）→ 最后才动精度（量化）。

1.5 用三个数字建立"手感"

在往下读之前，先记住三个数字，它们会在后面反复出现（全部可由第 10、13、14 章的公式复算）：

场景	算式	结果	含义
7B 模型推理权重（FP16）	6.74e9 × 2 B	≈ 13.4 GB	一张 24GB 消费卡装得下
7B 模型训练静态显存（BF16 + AdamW）	2N + 2N + 12N	≈ 107.2 GB	单张 80GB 卡装不下，必须分布式
7B 模型 KV Cache（seq=4096, batch=16, FP16, MHA）	2×L×h_kv×d_k×s×b×2 B	≈ 32 GB	比权重还大，这是推理显存规划的主战场

这三个数字合起来解释了整条学习路线的存在理由：**权重不大，但训练态和 KV Cache 很大**——所以需要分布式训练（第 15 章）和推理优化（第 11 章）。

> 数字口径提醒：第 14 章会给出另一种训练显存口径「每参数 18 B」，得到 7B ≈ 126 GB；第 10 章给出「每参数 16 B」，得到 107.2 GB。差别在于**梯度是否用 FP32 保存**。两个口径在原文不同文章里都出现过，做估算时先声明口径，不要混用。

第 2 章 编程语言基础：Python 的性能真相、C/C++ 的内存与 Linux 工具链

AI Infra 处在算法、框架、编译器、操作系统和硬件的交界处。同一个问题往往要在多种语言和工具之间来回切换。AIInfraGuide 第 1 章把这个分工拆成三层：

层次	常用技术	优势	典型工作
编排层	Python、Shell	开发快、生态丰富	模型定义、训练脚本、任务编排、数据处理
系统层	C/C++	可控的内存与性能、易连接底层 API	框架核心、算子调度、通信库、Python 扩展
设备层	CUDA C++、Triton	显式表达 GPU 并行	Kernel、算子融合、访存优化

三层不是替代关系。一行 `torch.matmul(x, w)` 的实际调用链是：

```
Python API
  └ 算子分发器（C++，ATen/dispatcher）
      └ cuBLAS / 自定义 CUDA Kernel
          └ GPU 指令与显存访问
```

原文由此给出一个很重要的能力定位：**AI Infra 工程师最重要的语言能力不是"背语法"，而是沿调用链跨层追踪问题。**

2.1 Python 名字绑定：一切"诡异 Bug"的源头

Python 变量更准确的说法是"名字绑定到对象"。赋值通常不复制对象，只建立新引用：

```
a = [1, 2]
b = a
b.append(3)
print(a)        # [1, 2, 3]
print(a is b)   # True：同一个对象
print(a == b)   # True：值相等
```

要区分两个运算：`is` 比较对象身份（适合写 `x is None`），`==` 调用值比较。

函数参数同样是名字绑定——既不是传统意义的"按值传递"，也不是"按引用传递"：

```
def mutate(xs: list[int]) -> None:
    xs.append(1)          # 修改调用方可见的同一个列表

def rebind(xs: list[int]) -> None:
    xs = [1, 2]           # 只让局部名字 xs 指向新列表

data: list[int] = []
mutate(data)
print(data)               # [1]
rebind(data)
print(data)               # 仍然是 [1]
```

这套模型能解释大量常见 Bug：默认可变参数、浅拷贝、缓存污染、多线程共享状态。

2.1.1 默认可变参数：函数定义时求值

```
# 错误：同一个列表会被多次调用共享
def collect_bad(x: int, result: list[int] = []) -> list[int]:
    result.append(x)
    return result

# 正确：用 None 表示"未提供"
def collect(x: int, result: list[int] | None = None) -> list[int]:
    if result is None:
        result = []
    result.append(x)
    return result
```

关键点：**默认参数在函数定义时求值一次，不是每次调用时求值。**

2.1.2 浅拷贝与深拷贝

```
import copy
src = [[1], [2]]
shallow = src.copy()          # 只复制最外层列表
deep = copy.deepcopy(src)     # 递归复制内部对象
shallow[0].append(9)
print(src)                    # [[1, 9], [2]]
print(deep)                   # [[1], [2]]
```

【推断】在 AI Infra 语境下，这个坑最常出现在"配置字典"和"缓存"上：把一个 config dict 浅拷贝后改嵌套字段，会污染原始 config，导致复现实验时结果对不上。

2.3 GIL 到底限制了什么

这是最容易被背错的知识点。原文的表述非常克制，值得逐字读：

> 常见 CPython 构建使用全局解释器锁（GIL）保护解释器内部状态。同一进程中，通常只有一个线程能同时执行 Python 字节码。因此，两个执行纯 Python CPU 循环的线程一般不能获得两倍速度。

但**这不等于"Python 线程没有用"**：

- 阻塞 I/O 时解释器会释放 GIL；
- NumPy、PyTorch 等原生扩展可在耗时计算时释放 GIL；
- 后台日志、预取和轻量控制任务常适合线程。

原文还特别提醒：**GIL 是 CPython 实现细节**，不应泛化成所有 Python 实现或所有构建的永恒规则。判断性能时要测量当前解释器和依赖，而不是只背结论。

【补充】这也解释了一个反直觉现象：在 DataLoader 里把 `num_workers` 从 0 调到 4，加速往往来自"子进程"而非"线程"；但当一个 batch 的预处理中有大量 NumPy 向量化操作时，即使只用线程也能看到收益，因为那些操作在 C 层释放了 GIL。

2.4 并发模型选择表

原文给了一张"先判断瓶颈再选模型"的表，这是实战中最省时间的一张表：

任务类型	典型例子	常见选择
I/O 密集	请求服务、读取大量小文件	线程或 asyncio
Python CPU 密集	纯 Python 解析、复杂循环	多进程或原生扩展
原生算子密集	NumPy/PyTorch/CUDA 运算	由底层线程池/GPU 并行，Python 负责调度
大数据跨进程	数据加载、共享缓存	多进程 + 共享内存，谨慎序列化

还要先分清两个词：**并发（concurrency）**是多个任务在重叠的时间段内推进；**并行（parallelism）**是多个任务在同一时刻真正执行。asyncio 是并发但不是并行。

多进程的主要成本（原文列举，实战中全都会遇到）：

- 创建进程和独立解释器；
- 参数与返回值的序列化/反序列化；
- 复制或映射内存；
- 进程间通信与同步。

以及一条 GPU 专属警告：**CUDA 上下文和多进程启动方式组合不当会引发初始化错误或额外显存占用**；使用框架的数据加载与分布式能力时，应遵循框架推荐的启动器和 start method，**不要在父进程初始化 CUDA 后随意 fork**。

2.5 基准测试与 profiling：从"慢"定位到具体代码

性能测试最常见的错误是只测一次、把初始化算进去，或者忘记 GPU 异步执行。正确的最小做法：

```
from statistics import median
from time import perf_counter

def benchmark(fn, warmup: int = 5, repeat: int = 20) -> float:
    for _ in range(warmup):
        fn()
    samples = []
    for _ in range(repeat):
        start = perf_counter()
        fn()
        samples.append(perf_counter() - start)
    return median(samples)
```

对于 GPU：**Kernel 启动通常是异步的**。CPU 计时器只量到"提交任务"的时间，需要在边界 `torch.cuda.synchronize()`，或使用 CUDA Event。而且同步会改变流水线行为，所以**端到端吞吐和单算子延迟要分别测量**。

原文建议的定位顺序是"由粗到细"，这条路径值得固化成肌肉记忆：

层级	手段	回答什么问题
端到端计时	墙钟时间	回归真实存在吗
阶段计时	分段打点	时间在数据、计算、通信还是保存
函数级 profiling	`cProfile`	哪个函数累计耗时高
行级/采样 profiling	`line_profiler`、`py-spy`	热点行 / 原生调用栈
内存 profiling	`tracemalloc`	Python 分配在哪里增长

层级	手段	回答什么问题
GPU profiling	torch.profiler、Nsight Systems/Compute	GPU 在等谁、kernel 是什么瓶颈

```
python -m cProfile -o profile.out train.py
python -m pstats profile.out
# pstats 内：sort cumulative → stats 30
```

2.6 Python 性能优化的优先级

原文给了一个非常"反直觉但正确"的顺序：

1. 先改算法复杂度和数据结构；
2. 减少不必要的 I/O、复制和序列化；
3. 把细粒度 Python 循环改成批处理/向量化原生算子；
4. 减少 Python 与 C++/GPU 边界的频繁往返；
5. 确认热点后，再写 C++、CUDA 或 Triton 扩展。

原文的例子很到位：**一百万次小算子调用即使每次都很快，也会累积大量调度开销**。把数据组成批次后调用一次大算子，通常比微调 Python 语法有效得多。

【推断】这条优先级在 AI Infra 里等价于一条工程守则：**先怀疑"调用次数"，再怀疑"单次调用效率"**。Kernel launch overhead、Python 分发开销、`.item()` 触发的隐式同步，都属于"次数"问题。

2.7 C/C++：值、指针、引用与生命周期

这是 CUDA 宿主端代码的阅读门槛。原文的切入点是"值、地址、指针与引用"——不要求精通模板元编程，但要能读懂指针、内存管理、编译链接过程。

概念	本质	常见误用
值（value）	一块有类型的内存	大对象按值传参导致拷贝
地址（address）	内存编号	保存地址但对象已析构（悬垂）
指针（pointer）	存地址的变量，可为 `nullptr`	不检查空、越界、double free
引用（reference）	对象的别名，不可重新绑定	返回局部变量的引用

对象存储期（storage duration）决定"什么时候生、什么时候死"：自动存储期（栈）、静态存储期、动态存储期（堆）、线程存储期。**生命周期错配是 C/C++ 崩溃的第一大类原因。**

2.7.1 RAII：把资源绑定到对象生命周期

RAII（Resource Acquisition Is Initialization）的核心是：**资源在构造时获取，在析构时释放**。这样即使中途`throw`，栈展开也会调用析构函数。

```
// RAII 的最小例子
class CudaBuffer {
public:
    explicit CudaBuffer(size_t bytes) { cudaMalloc(&ptr_, bytes); }
    ~CudaBuffer() { if (ptr_) cudaFree(ptr_); }
    CudaBuffer(const CudaBuffer&) = delete;             // 禁止拷贝
    CudaBuffer& operator=(const CudaBuffer&) = delete;
    void* get() const { return ptr_; }
private:
    void* ptr_ = nullptr;
};
```

2.8 编译与链接：一条必须背下来的流水线

```
源文件 .cpp
  └─ 预处理：展开 #include、宏和条件编译
      └─ 编译：语法/类型检查并生成汇编
          └─ 汇编：生成目标文件 .o
              └─ 链接：解析符号并生成可执行文件或库
```

可以逐级观察（原文给的命令，非常实用）：

```
g++ -E main.cpp -o main.ii     # 只预处理
g++ -S main.ii  -o main.s      # 生成汇编
g++ -c main.cpp -o main.o      # 生成目标文件
g++ main.o -o app              # 链接
```

2.8.1 三类错误不要混在一起

阶段	典型报错	常见原因
编译	`not declared`、类型不匹配	缺声明、模板实例化失败、语法错误
链接	`undefined reference`	只声明未定义、漏链接库、ABI/签名不一致
运行时装载	`cannot open shared object file`	动态库不在搜索路径、架构不匹配

定位符号与依赖的常用命令：

```
nm -C build/app | less      # 查看并反修饰 C++ 符号
ldd build/app               # 查看动态库依赖（Linux）
readelf -d build/app        # 查看 ELF 动态段
# macOS 对应：otool -L build/app
```

2.8.2 静态库、动态库与 ABI

- **静态库**（Linux `.a`）在链接时把所需代码并入目标文件，部署直接，但产物更大；
- **动态库**（Linux `.so`、macOS `.dylib`）在装载/运行时解析，可被多个程序共享，但要处理搜索路径与 ABI 兼容。

Linux 动态库搜索顺序受 RPATH/RUNPATH、`LD_LIBRARY_PATH`、系统缓存与默认目录影响。原文的提醒很重要：**临时设置 `LD_LIBRARY_PATH` 适合诊断，不应成为掩盖安装布局问题的长期万能方案。**

ABI（Application Binary Interface）规定二进制层面的调用约定、名称修饰、对象布局和标准库类型实现。以下情况都可能造成 ABI 不兼容：编译器或标准库不一致、编译选项改变对象布局、库升级后导出符号变化、Debug/Release 运行库混用、CPU 架构不同。

> 实战映射：Python C++ 扩展"能编译但 import 失败"，除了 Python 路径，**必须检查扩展和依赖库的 ABI、架构与符号**。PyTorch 自定义算子在不同 CUDA/PyTorch 版本间"编译成功但运行崩"，绝大多数是这个原因。

2.10 Linux 工具链：AI Infra 的日常工作面

2.10.1 文件、文本流与管道

```
pwd; ls -lah
mkdir -p runs/exp01/logs
cp -a configs runs/exp01/          # 保留属性复制
find runs -type f -name '*.log'
rg 'CUDA error|out of memory' runs
python train.py >train.log 2>train.err   # 分别重定向
python train.py >train.all.log 2>&1      # 合并
rg 'loss=' train.log | tail -n 20
```

2.10.2 引号、变量与退出码

```
name='experiment 01'
printf '%s\n' "$name"    # 双引号：保留为一个参数
printf '%s\n' '$name'    # 单引号：不展开
```

未加引号的变量可能发生单词分割和通配符展开，**Shell 脚本中几乎总应写 `"$variable"`**。每个命令都有退出码（0 成功，非零失败），健壮脚本的标准开头：

```
#!/usr/bin/env bash
set -euo pipefail
```

`-e` 未处理失败即退出；`-u` 使用未定义变量报错；`pipefail` 管道中任一命令失败则整体失败。

2.10.3 进程、信号与远程会话

```
ps -ef | rg train.py
pgrep -af train.py
python train.py &        # 后台
jobs; fg %1
kill -TERM <pid>         # 请求优雅退出（优先）
kill -KILL <pid>         # 内核立即终止，无法清理（最后手段）
tmux new -s train        # Ctrl-b d 脱离；tmux attach -t train 回来
```

原文的提醒很关键：**优先 SIGTERM，让程序有机会保存 checkpoint、释放共享资源和刷新日志**。长时间训练任务不要只依赖 SSH 终端。

2.10.4 资源排查命令表

问题	常用命令
CPU/负载	`top`、`uptime`、`mpstat`
进程内存	`ps`、`top`、`pmap -x PID`
系统内存	`free -h`、`vmstat`
磁盘容量	`df -h`
目录占用	`du -sh PATH`、`du -h --max-depth=1`
文件 I/O	`iostat`、`iotop`
监听端口	`ss -lntp`
连接测试	`curl -v URL`、`nc -vz HOST PORT`

原文特别指出："内存不够"至少可能指**主机 RAM、GPU 显存、共享内存 `/dev/shm`、进程地址空间限制或磁盘满**。先确认资源种类再行动。

> 【补充】`/dev/shm` 这一项在 PyTorch 里非常真实：DataLoader 用共享内存传 batch，容器默认 `/dev/shm` 只有 64MB 时会报 `Bus error` 或 `DataLoader worker exited unexpectedly`，通常要用 `--shm-size` 放大。

2.10.6 GPU 软件栈：四个容易混淆的部分

```
应用 / PyTorch / 自定义扩展
   |
CUDA Runtime 与数学库（cudart、cuBLAS、cuDNN 等）
   |
用户态驱动库（libcuda）
   |
内核态 NVIDIA Driver
   |
GPU
```

组件	是什么	怎么看
NVIDIA Driver	操作系统与 GPU 通信的基础	`nvidia-smi` 主要反映驱动状态
CUDA Toolkit	开发工具集合（nvcc、头文件、分析工具、库）	`nvcc --version`、`which nvcc`
CUDA Runtime/数学库	应用运行时加载的用户态组件	框架自带或系统安装
cuDNN	面向深度学习算子的独立加速库	不等于整个 CUDA

原文有一条极易踩错的事实：**`nvidia-smi` 显示的 "CUDA Version" 通常表示驱动可支持的最高 CUDA 兼容级别，并不证明本机安装了同版本 nvcc。**

诊断 GPU 环境的固定顺序（建议抄下来）：

```
nvidia-smi                                    # 1 驱动是否看到设备
which nvcc && nvcc --version                  # 2 编译工具链来自哪里
which python && python -c 'import sys; print(sys.executable)'  # 3 解释器来自哪里
python -c 'import torch; print(torch.__version__, torch.version.cuda, torch.cuda.is_availab
```

第 3 章 数学基础：形状、概率、梯度与数值稳定性

AIInfraGuide 第 2 章给数学基础定了一个非常工程化的目标：**不要求每天手推复杂定理，但要求看到公式时能立刻回答三个工程问题。**

3.1 读公式先问的三个问题

原文以语言模型输出投影为例：

```
Y = XW,   X ∈ R^{(BS)×H},   W ∈ R^{H×V}
```

问题	答案	工程含义
① 形状是否匹配	内维 H 相同，输出 (BS)×V	决定这个算子能不能写、要不要转置

问题	答案	工程含义
② 代价是多少	2·B·S·H·V FLOPs	决定它是 compute bound 还是 memory bound
③ 实现风险是什么	权重很大、logits 很大	数据类型、内存布局、分块、并行、Softmax 稳定性

这三问贯穿整个 AI Infra 技术栈。后面每一章都会回到它们。

3.2 形状直觉：(B,S,H) × (H,V) 到底发生了什么

先固定一套 batch-first 记号（原文 1.1 节）：

符号	含义
B	batch size
S	sequence length
H	hidden size
V	vocabulary size
N_h	attention head 数
D_h = H / N_h	每个 head 的维度

3.2.1 从 (B,S,H) 到 (B,S,V)

```
X  : (B, S, H)
W  : (    H, V)
------------------------------
Y  : (B, S, V)      ← 内维 H 被"消掉"（归约维）
```

记忆口诀：矩阵乘法 = 把内维消掉，把外维留下。 看到 `(B,S,H) @ (H,V)`，先划掉两个 H，剩下 `(B,S,V)`。这个动作练到条件反射，读任何模型代码都不会卡在 shape 上。

3.2.2 同一套口诀推 6 个常见算子

算子	输入 A	输入 B	输出	归约维
QKV 投影	(B,S,H)	(H,H)	(B,S,H)	H
Attention 分数	(B,N_h,S,D_h)	(B,N_h,D_h,S)	(B,N_h,S,S)	D_h
Attention 输出	(B,N_h,S,S)	(B,N_h,S,D_h)	(B,N_h,S,D_h)	S
FFN 升维	(B,S,H)	(H,d_ff)	(B,S,d_ff)	H
FFN 降维	(B,S,d_ff)	(d_ff,H)	(B,S,H)	d_ff

算子	输入 A	输入 B	输出	归约维
LM Head	(B,S,H)	(H,V)	(B,S,V)	H

【推断】把这 6 行背下来，你就拿到了"读 Transformer 代码不迷路"的最小地图：整个前向过程就是这 6 个矩阵乘的排列组合，加上几个逐元素算子（Softmax、LayerNorm、激活）和几个 reshape/transpose。

3.2.3 shape 相同不等于内存布局相同

原文的一个重要提醒：工程中的"张量"还带有数学 shape 之外的信息——**数据地址 + shape + stride + dtype + device + layout**。

```
shape (2,3) 的行主序矩阵          底层连续存储 [1,2,3,4,5,6]
X = [[1,2,3],                      以元素为单位的 stride = (3,1)
     [4,5,6]]                      线性偏移 offset(i,j) = 3i + j

转置视图 X^T: shape (3,2), stride (1,3), 通常无需复制数据
               但按转置后的最后一维扫描时，访问不再连续
```

所以看到 `view`、`reshape`、`transpose`、`contiguous` 时，要**同时思考逻辑维度和物理布局**。这也是性能坑的高发区：`transpose` 之后直接送进 kernel，可能触发隐式 copy 或非合并访存（第 14 章）。

3.3 Softmax 的概率解释

3.3.1 logits 不是概率

模型最后一层输出 V 个任意实数 `z = (z_1..z_V)`，叫 **logits**。它们可以为负，也不要求和为 1。Softmax 把它们转换为分类分布：

```
p_i = softmax(z)_i = exp(z_i) / Σ_j exp(z_j)
```

显然 `p_i > 0` 且 `Σ p_i = 1`。

概率解释：Softmax 是一个"以 logits 为能量的 Boltzmann 分布"。`exp` 把实数轴映射到正数，归一化把总量压成 1。logits 之间的**差值**决定概率比：

```
p_i / p_j = exp(z_i − z_j)
```

这条式子解释了两件事：① 只有差值有意义，所以加一个常数不改变结果；② logit 差 1 意味着概率比是 e ≈ 2.718 倍。

3.3.2 平移不变性 = 数值稳定的依据

```
softmax(z + c) = softmax(z)      因为分子分母都乘了 exp(c)
```

取 `c = -max(z)`，最大的指数输入变成 0，`exp` 结果最大为 1，不会溢出：

```
def stable_softmax(x):
    maximum = max(x)
    exps = [exp(value - maximum) for value in x]
    denominator = sum(exps)
    return [value / denominator for value in exps]
```

原文的细致之处：减最大值防正向溢出，但**很小的项仍可能下溢到 0**。这通常代表它相对最大项确实可以忽略；但如果后续还要取对数，应直接用稳定的 `log_softmax`，避免"先变成 0 再 log(0)"。

3.3.3 LogSumExp 与融合

```
LSE(z) = log Σ_j exp(z_j) = m + log Σ_j exp(z_j - m),   m = max_j z_j
log softmax(z)_i = z_i - LSE(z)
```

交叉熵实现通常融合 `log_softmax + NLLLoss`，既减少中间张量和访存，也避免不稳定的"先 Softmax 再取 log"。**这就是"数值稳定的算法往往也更快"的第一个例子**（第 3.7 节还会再遇到）。

3.3.4 温度参数

```
p_i(T) = exp(z_i / T) / Σ_j exp(z_j / T)
```

温度	效果	用途
T < 1	放大 logit 差异，分布更尖锐	更确定、更少发散
T = 1	原始分布	默认
T > 1	缩小差异，分布更平坦	更多样
T → 0^+	逐渐接近 argmax（贪心）	确定性输出
T → ∞	逐渐接近均匀分布	纯随机

原文的提醒：**温度改变的是采样分布，不等于修改模型权重**；实现应先缩放 logits，再应用稳定 Softmax。

3.4 交叉熵：为什么它就是负对数似然

3.4.1 定义与 one-hot 化简

真实分布 q 与模型分布 p 的交叉熵：

```
H(q,p) = -Σ_i q_i log p_i
```

若标签是 one-hot（正确类别 y 处为 1，其余为 0），则：

```
L = -log p_y
```

模型给正确类别越高概率，损失越小。`p_y = 1` 时损失 0；`p_y → 0` 时损失很大。**所以最小化交叉熵 = 最大化正确类别的对数概率 = 最大似然。**

3.4.2 熵、KL 与交叉熵的关系

```
H(p)     = -Σ_i p_i log p_i          (分布自身的熵，衡量不确定性)
D_KL(q‖p)=  Σ_i q_i log(q_i / p_i)   (从 q 到 p 的 KL 散度)
H(q,p)   =  H(q) + D_KL(q‖p)         (分解式)
```

三个关键性质：

- `D_KL ≥ 0`，两分布相同时为 0；
- KL **不对称**：`D_KL(q‖p) ≠ D_KL(p‖q)`，所以它不是严格意义的距离；
- 训练时真实分布 q 固定，`H(q)` 与模型参数无关，**所以最小化交叉熵等价于最小化 KL 散度**。

熵的单位由对数底决定：自然对数对应 nat，以 2 为底对应 bit。机器学习损失通常用自然对数。

3.4.3 Perplexity

```
PPL = exp(L̄)     L̄ 为平均 token 负对数似然
```

直觉：模型在每一步面对的"有效候选数"。但原文提醒：**不同 tokenizer、数据预处理、上下文长度、是否忽略特殊 token 都会影响 PPL，不能脱离评测设置直接横比。**

3.5 链式法则与反向传播

3.5.1 链式法则

若 `y = f(u), u = g(x)`，则 `dy/dx = (dy/du)(du/dx)`。多层网络就是复合函数：

```
h1 = f1(x),  h2 = f2(h1),  L = f3(h2)
∂L/∂x = (∂L/∂h2)(∂h2/∂h1)(∂h1/∂x)
```

概念上是 Jacobian 连乘；实际自动微分不会显式创建巨大 Jacobian，而是高效计算 vector–Jacobian product（反向模式）。

3.5.2 计算图：为什么梯度要"累加"

原文的例子非常精炼。设：

```
a = x*y,   b = a + x,   L = b²
```

反向从 `∂L/∂L = 1` 开始：`∂L/∂b = 2b`；因为 `b = a + x`，所以 `∂L/∂a = 2b`，`∂L/∂b · ∂b/∂x = 2b`。关键在于 **x 有两条路径影响 L**：

```
∂L/∂x = ∂L/∂b·∂b/∂x + ∂L/∂a·∂a/∂x = 2b + 2by
∂L/∂y = 2bx
```

这解释了计算图中一个张量被多处使用时为什么要累积梯度——也就是 PyTorch 里 `x.grad` 是累加而不是覆盖的根本原因（第 13 章会看到 `zero_grad()` 的必要性）。

3.5.3 线性层的反向：三行 shape 检查

采用 batch 行向量约定 `Y = XW + b`，其中 `X ∈ R^{B×d_in}`、`W ∈ R^{d_in×d_out}`。给定上游梯度 `G = ∂L/∂Y ∈ R^{B×d_out}`：

```
∂L/∂X = G Wᵀ    ∈ R^{B×d_in}
∂L/∂W = Xᵀ G    ∈ R^{d_in×d_out}
∂L/∂b = Σ_i G_i,:  ∈ R^{d_out}
```

用 shape 自检（这是最快的验算方式）：

```
dX: (B, d_out) @ (d_out, d_in)  -> (B, d_in)     ✓
dW: (d_in, B)  @ (B, d_out)     -> (d_in, d_out) ✓
db: 对 batch 维归约              -> (d_out,)       ✓
```

原文的结论：**一个线性层的反向通常包含两次 GEMM 和一次归约**，这也是训练算力显著高于单次前向的原因（约 3 倍，见第 10 章 6Ns 公式）。

3.5.4 Softmax + 交叉熵：p − y

这是深度学习最重要的简化之一。one-hot 标签 y，损失 `L = -Σ y_i log p_i`，与 Softmax 联合求导后：

```
∂L/∂z_i = p_i - y_i
```

对正确类别梯度是 `p_y - 1`（负），对其他类别是 `p_i`（正）。**预测越错，推动 logits 修正的力度越大。**

这条式子也是"融合 kernel 能省一次读写"的依据：只要拿到 p 和 y 就能直接算梯度，不需要物化中间概率再反传。

3.6 混合精度的数值问题

3.6.1 浮点数是有限集合

```
(-1)^s × significand × 2^exponent
```

指数位决定动态范围，尾数位决定相邻可表示数的精细程度。 这两件事必须分开理解：

格式	总位数	指数位	尾数字段位	主要特点
FP32	32	8	23	范围和精度较均衡
TF32	19（乘法输入语义）	8	10	保留 FP32 范围，降低乘法精度，常用 FP32 累加
FP16	16	5	10	精度尚可但动态范围较小
BF16	16	8	7	接近 FP32 动态范围，精度低于 FP16
FP8 E4M3	8	4	3	精度优先、范围较小
FP8 E5M2	8	5	2	范围优先、精度更低

（表中不把隐含前导位计入尾数字段；TF32 通常是 NVIDIA Tensor Core 的计算模式，不是常规内存存储 dtype。）

3.6.2 精度与动态范围是两件事

- FP16 尾数字段比 BF16 多，同一数量级下能表示更细的差别；但 FP16 指数位少，**最大有限值只有 65504**，非零数的范围也窄得多。
- BF16 牺牲精度换成与 FP32 相同的指数位数，训练中通常更不易溢出/下溢；但它仍有明显舍入误差，**尤其在把很小增量加到很大数上时**。

这直接回答了那个经典面试题："BF16 和 FP16 都是 16 位，为什么大模型训练更偏爱 BF16？"——**因为 BF16 的动态范围接近 FP32，大多数情况下可以不做 Loss Scaling。**

3.6.3 舍入、非结合性、归约顺序

```
>>> 0.1 + 0.2 == 0.3
False
```

浮点间距随数值绝对值增大而增大。若 `|δ|` 远小于当前 x 附近的可表示间距，则 `fl(x + δ) = x`——**这就是优化器必须保留 FP32 master weights 的原因：低精度权重上过小的更新会被直接舍掉。**

加法不满足结合律：

```
fl(fl(a+b)+c) ≠ fl(a+fl(b+c))
```

所以并行归约用树形加法时，**不同线程数、block 划分和原子操作顺序可能产生略不同结果**。原文的判断很重要：**非确定性不一定代表实现错误**，但必须在可接受误差内，并符合任务对复现性的要求。

提高归约精度的手段：更高精度累加、成对/树形求和、Kahan 补偿求和、先局部归约再合并部分和。

3.6.4 溢出、下溢、NaN 与灾难性消减

现象	定义	常见源头
overflow	绝对值超过最大有限值，变成 ±∞	大 logits、大学习率
underflow	太接近 0，进入 subnormal 或舍入为 0	FP16 小梯度
NaN	0/0、∞−∞ 或非法运算	mask 全被屏蔽、除零、负数开方
灾难性消减	两个接近的大数相减，有效位抵消	`E[X²] − E[X]²` 算方差

原文给的排查建议很实用：**排查 loss 变 NaN 时，应找到第一个非有限张量，而不是只盯最终 loss。**

灾难性消减的经典案例是用 `Var(X) = E[X²] − E[X]²` 算方差：两项都很大且接近，相减后精度损失严重。更稳的方法是 two-pass variance 或 Welford 在线算法：

```
n ← n+1
δ = x_n − μ_{n−1}
μ_n = μ_{n−1} + δ/n
M2,n = M2,n−1 + δ(x_n − μ_n)
总体方差 = M2,n / n
```

Welford 还能合并不同分块的统计量，**适合并行归约**——第 9 章会看到它是 LayerNorm kernel 的标准做法。

3.6.6 混合精度训练的基本模式

原文的定义非常准确："混合精度不是把所有东西都换成 FP16"。常见模式是：

环节	精度策略
大型矩阵乘输入	FP16/BF16，走 Tensor Core
乘加累加	FP32 或更高的指定累加精度
归约/归一化/Softmax 等敏感操作	保留或内部提升到 FP32
优化器状态 / master weights	FP32
输出 dtype	按硬件和算子选择

原文的告诫：**每个框架和硬件的具体策略不同，应查实际算子文档和 profiler，而不是从输入 dtype 猜内部计算路径。**

3.6.7 Loss Scaling 为什么有用

反向传播中的 FP16 小梯度可能下溢为 0。令缩放因子为 S：

```
L' = S·L          ⇒          ∇L' = S·∇L
反向完成后：∇L = ∇L' / S
```

动态 Loss Scaling 的循环：

1. 用当前 scale 计算反向；
2. 检查梯度是否含 inf/NaN；
3. 若溢出，跳过更新并减小 scale；
4. 连续稳定若干步后尝试增大 scale。

原文的边界条件：**Loss Scaling 主要解决小梯度下溢，scale 过大反而会造成溢出**；BF16 动态范围更大，通常不依赖 Loss Scaling，但仍可能因模型本身不稳定产生非有限值。

3.7 综合算例：把数学变成工程判断

原文第 9 节给了 7 个算例，这里保留最能形成"手感"的 2 个（其余 5 个的结论已分散在第 5、10、11 章）。

3.7.1 为什么除以 √D_h

假设 q_i、k_i 独立、均值 0、方差 1：

```
qᵀk = Σ_{i=1..D_h} q_i k_i
每项期望约 0、方差约 1  ⇒  和的方差约 D_h，标准差约 √D_h
除以 √D_h 后：Var(qᵀk / √D_h) ≈ 1
```

避免维度增大时 logits 过度扩张，Softmax 进入极端饱和区（梯度趋近 0）。

3.7.2 Online Softmax 的分块合并

这是 FlashAttention 的数学核心，务必记住这两个递推式。已处理部分的状态为 `(m, l)`，新 block 的状态为 `(m_b, l_b)`：

```
m_new = max(m, m_b)
l_new = exp(m − m_new)·l + exp(m_b − m_new)·l_b
```

若同时维护加权 Value 累加器，还能避免保存完整注意力矩阵。**这就是"数值稳定的算法天然可融合"的第二个例子。**

第 4 章 Transformer 为什么是 AI Infra 的"通用底座"

这一章开始进入本笔记的重心。**Transformer 部分共 9 章（第 4～12 章）**，原因是 AIInfraGuide 的学习路线里，后续每一层都在围绕它做优化：CUDA 层优化它的算子、分布式层切分它的参数、推理层加速它的生成。不理解它，后面所有优化都变成"背结论"。

4.1 AI Infra 四类日常工作，全部指向 Transformer

原文第 3.1 章把 AI Infra 的日常工作归纳为四类，并逐条说明它们为什么必须懂 Transformer：

工作方向	具体任务	需要 Transformer 的哪部分知识
CUDA 算子开发	写 FlashAttention、优化 GEMM、算子融合	Self-Attention 的 QK^T → softmax → PV 流程与访存模式
分布式训练	TP / PP / DP、ZeRO、3D 并行	切分点由结构决定：TP 沿 Attention 多头维切，PP 沿 Block 堆叠方向切
推理部署	KV Cache、Continuous Batching、量化、投机解码	KV Cache 从哪来、为什么随 S 线性增长
性能分析与调优	Nsight / torch.profiler 定位瓶颈	看到 kernel 名字要知道它对应哪个模块

原文的一句话很到位：**"看到一个耗时很长的 kernel 名称，连它在做什么都搞不清楚"**——这是不懂模型结构的人做性能优化时最常见的困境。

4.2 RNN 的两个致命弱点与 Transformer 的两次范式转换

在 Transformer 之前，序列建模主流是 RNN/LSTM。原文指出两个致命弱点：

弱点	机制	后果
无法并行训练	第 t 步依赖第 t-1 步输出，必须串行	2048 token 要顺序执行 2048 步，GPU 数千核心闲置
长距离依赖遗忘	信息逐步传递会衰减	数千 token 以上效果明显下降

2017 年 Vaswani 等人的 *Attention Is All You Need* 用 Self-Attention 完全取代循环结构，带来两个根本优势：

- **天然并行性**：所有 token 对的注意力权重可以同时算，本质是一次矩阵乘 QK^T；
- **全局信息访问**：任意两个位置一步直达，不存在距离衰减。

代价是 **$O(N^2)$** 复杂度。原文的判断很关键：在实际训练场景中，并行带来的加速远超 $O(N^2)$ 的额外计算；而 $O(N^2)$ 带来的显存与计算问题，**恰恰成为 AI Infra 优化的核心战场**（FlashAttention、KV Cache 管理）。

4.3 三条分岔路：为什么 Decoder-only 赢了

原始 Transformer 是 Encoder-Decoder 架构。此后演化出三条路线：

架构	代表模型	核心特点	典型任务
Encoder-only	BERT、RoBERTa	双向注意力，看到完整上下文	分类、NER、句子相似度
Encoder-Decoder	T5、BART、mBART	Encoder 双向理解 + Decoder 自回归生成	翻译、摘要、问答
Decoder-only	GPT 系列、LLaMA、Mistral、Qwen	单向因果注意力，自回归生成	通用文本生成、对话、推理

原文用了三个很形象的类比：Encoder-only 像阅读理解专家（会读不会写）；Encoder-Decoder 像翻译官（先理解再逐句翻译）；Decoder-only 像即兴演讲者（边想边说，只基于已说过的内容）。

4.3.1 Decoder-only 胜出的四条理由

理由	展开
统一的训练范式	目标只有"预测下一个 token"，数万亿无标注文本可直接用；Encoder-Decoder 要设计输入-输出对
扩展更简单	只有一种 Block 堆叠，扩规模只需加层数或加宽度；Encoder-Decoder 要同时扩展两个组件并平衡比例
推理效率优势	只需维护一套 KV Cache；Encoder-Decoder 要维护 Encoder 侧输出 + Decoder 侧 KV Cache 两套
涌现能力的经验观察	参数超过一定阈值后，few-shot / zero-shot 与 CoT 能力更明显

原文的总结一句话：**Decoder-only 在大规模场景下，以最简洁的架构获得了最好的通用能力，同时对工程系统最友好。**

4.4 三种注意力机制的分工

原文第 3.2 章提醒了一个容易被忽略的点：原始架构里其实有**三种**注意力，各自职责不同：

机制	Q 来自	K/V 来自	是否带 mask	作用
Encoder Self-Attention	Encoder 输入	Encoder 输入	无（双向）	让输入内部互相理解
Decoder Masked Self-Attention	Decoder 输入	Decoder 输入	有（因果）	只能看已生成的 token
Cross-Attention	Decoder	Encoder 输出	无	把输入信息喂给生成过程

Decoder-only 砍掉了 Cross-Attention，只留 Masked Self-Attention + FFN。 这就是为什么大模型的 Block 结构可以那么简单——简单到可以画成一张纸（第 10 章）。

4.5 端到端数据流（一页纸版本）

这是本笔记的"总图"，后面 8 章都是它的放大镜：

```
文字 "猫在桌子上"
   │ ① Tokenization（第 12 章）
   ▼
token ids [3421, 528, 11082, 319]            shape (S,)
   │ ② Embedding 查表 W_emb[V, d]（第 12 章）
   ▼
X: (B, S, d_model)                           语义信息
   │ ③ 位置编码（第 8 章）——RoPE 作用在 Q/K 上，Sinusoidal 加在输入上
   ▼
┌──────────────────────── × L 层（第 10 章） ────────────────────────┐
│  h = x + MaskedMHA(RMSNorm(x))        ← 第 5、6、9 章             │
│  y = h + SwiGLU_FFN(RMSNorm(h))       ← 第 7、9 章                │
└──────────────────────────────────────────────────────────────────┘
   │ ④ 最终 RMSNorm
   ▼
hidden: (B, S, d_model)
   │ ⑤ LM Head: (d_model) × (d_model, V)
   ▼
logits: (B, S, V)
   │ ⑥ 取最后位置 → Softmax / 采样（第 11 章）
   ▼
next token id → 追加回序列，循环
```

第 5 章 Self-Attention：QK^T → scale → mask → softmax → PV 与 $O(N^2)$ 的来源

5.1 历史脉络：Attention 是怎么长成现在这样的

原文把演进拆成三步，每一步都在解决前一步的效率问题：

年份	方案	关键改进	遗留问题
2014	Bahdanau Attention	不再把输入压成固定向量，Decoder 每步"回看"Encoder 所有隐状态	匹配函数是一个额外前馈网络（additive），计算效率低
2015	Luong Attention	用点积代替前馈网络衡量相似度	仍是跨序列 Attention
2017	Self-Attention	完全抛弃 RNN，序列内部自关注	$O(N^2)$ 复杂度

原文强调了 Luong 那一步为什么关键：**点积可以被表示为矩阵乘法，天然适合 GPU 并行加速**。这个"看似简单的改进，为后来 Self-Attention 的大规模应用铺平了道路"。

5.2 从信息检索理解 Q、K、V

这是全书最好用的一个类比，值得复述：

> 你走进图书馆，脑子里有个模糊需求"我想了解并行计算"——这是 **Query**。每本书书脊上的标签是 **Key**。你拿 Query 和每个 Key 比对，得到匹配分数。然后你不是把标签拿走，而是按分数加权汇总每本书的**实际内容（Value）**。

三者的形式化定义（`X` 形状 `(N, d_model)`）：

```
Q = X · W_Q     "我需要什么信息"
K = X · W_K     "我能提供什么信息的线索"
V = X · W_V     "我实际携带的信息"
```

为什么必须从同一个 X 做三次不同投影，而不是直接用 X？ 原文的解释很精准：一个 token "需要什么"和它"能提供什么"往往是不同的——动词需要关注主语和宾语（Query 方向），但它作为被关注对象时提供的是动作语义（Key/Value 方向）。三个独立投影给模型自由度去学习这些不同映射。

5.3 五步计算：每一步的 shape 都要能背

输入 `X: (N, d_model)`，以单头为例，`W_Q/W_K/W_V` 形状都是 `(d_model, d_model)`。

第 1 步：线性投影

```
Q = X·W_Q :  (N, d_model) × (d_model, d_model) = (N, d_model)
```

原文的工程细节：实际实现中会把 `W_Q, W_K, W_V` 合并成一个大矩阵 `W_QKV: (d_model, 3·d_model)`，**做一次 GEMM 然后 split**，这样能更好利用 GPU 算力。这是"融合"思想在 Attention 里的第一个体现。

第 2 步：计算原始注意力分数

```
S = Q·Kᵀ :  (N, d_model) × (d_model, N) = (N, N)
```

`S[i][j]` 是第 i 个 token 的查询与第 j 个 token 的索引的点积，代表 i 对 j 的原始关注程度。

这一步产生了 N×N 矩阵，这就是 Self-Attention 平方复杂度的根源。

第 3 步：缩放

```
S_scaled = S / √d_k
```

原文的一句话理由：`d_k` 较大时点积结果会变得很大，导致 Softmax 梯度趋近于零。严格推导见第 3 章 3.7.1（方差约 d_k，标准差约 √d_k，除完方差回到 1）。

注意这里的分母是 **`√d_k`（单头维度）**，不是 `√d_model`。这是一个高频写错点：两者只有在单头时才相等。第 5.4 节的 PyTorch 示例用了 `√d_model`，那是单头实现，刚好等价。

第 4 步：Softmax 归一化

```
A = softmax(S_scaled) ∈ R^{N×N},  每行和为 1
```

原文指出 Softmax 的双重作用：① 把任意实数映射到 (0,1) 可作为权重；② 保证每行和为 1，形成合法概率分布。

第 5 步：加权求和 + 输出投影

```
Output = A·V :  (N, N) × (N, d_model) = (N, d_model)
Final  = Output·W_O :  (N, d_model) × (d_model, d_model) = (N, d_model)
```

`W_O` 在多头场景下尤其重要——**它负责把多个头拼接后的表示重新混合**。

完整公式

```
Attention(Q, K, V) = softmax(QKᵀ / √d_k) · V
```

5.4 手写实现：单头与多头

单头实现（原文代码，可直接跑）：

```
import math
import torch
import torch.nn as nn
import torch.nn.functional as F

class SingleHeadSelfAttention(nn.Module):
    def __init__(self, d_model: int):
        super().__init__()
        self.d_model = d_model
        self.W_Q = nn.Linear(d_model, d_model, bias=False)
        self.W_K = nn.Linear(d_model, d_model, bias=False)
        self.W_V = nn.Linear(d_model, d_model, bias=False)
        self.W_O = nn.Linear(d_model, d_model, bias=False)

    def forward(self, x: torch.Tensor, mask: torch.Tensor = None):
        Q = self.W_Q(x)                                    # (B, S, d)
        K = self.W_K(x)
        V = self.W_V(x)
        scores = torch.matmul(Q, K.transpose(-2, -1))       # (B, S, S)
        scores = scores / math.sqrt(self.d_model)           # 单头时 d_model == d_k
        if mask is not None:
            scores = scores.masked_fill(mask == 0, float('-inf'))
        attn = F.softmax(scores, dim=-1)                    # (B, S, S)
        context = torch.matmul(attn, V)                     # (B, S, d)
        return self.W_O(context)                            # (B, S, d)
```

多头实现的关键只有两行 reshape/transpose：

```
# (B, S, d_model) -> (B, S, N_h, D_h) -> (B, N_h, S, D_h)
Q = self.W_Q(x).view(B, S, self.num_heads, self.head_dim).transpose(1, 2)
# ... K, V 同理；算完后
# (B, N_h, S, D_h) -> (B, S, N_h, D_h) -> (B, S, d_model)
out = out.transpose(1, 2).contiguous().view(B, S, -1)
```

> **`.contiguous()` 不是可选的。** `transpose` 只改 stride，`view` 要求内存连续，缺了它 PyTorch 会报错或触发隐式 copy。这是第 3 章"shape 相同 ≠ 布局相同"的直接后果。

5.5 Causal Mask：并行训练时如何"假装"是串行的

5.5.1 为什么需要

训练目标是"给定前面的 token 预测下一个"。计算第 i 个 token 的 Attention 时，它只能看位置 0..i，不能偷看 i+1 之后——否则预测变成"开卷考试"。

但训练时为了效率会把整个序列一次性送入并行计算。**并行意味着所有 token 的 Attention 同时算，不加限制每个 token 都会看到整个序列。** 因果掩码就是用来在并行计算的同时强制执行"只能看过去"。

5.5.2 掩码矩阵长什么样

序列长度 5，`S` 形状 (5,5)，`j > i` 的位置必须屏蔽（0 保留，−inf 屏蔽）：

```
Token:  t0    t1    t2    t3    t4
t0  [   0   -inf  -inf  -inf  -inf ]
t1  [   0     0   -inf  -inf  -inf ]
t2  [   0     0     0   -inf  -inf ]
t3  [   0     0     0     0   -inf ]
t4  [   0     0     0     0     0  ]
```

另一种"可见性"画法（1 可见，0 不可见），是一个下三角：

```
t0  [ 1 0 0 0 0 ]   t0 只能看自己
t1  [ 1 1 0 0 0 ]   t1 能看 t0 和自己
t2  [ 1 1 1 0 0 ]
t3  [ 1 1 1 1 0 ]
t4  [ 1 1 1 1 1 ]   t4 能看全部
```

加上掩码后，被屏蔽位置变成 `-inf`，`exp(-inf) = 0`，Softmax 权重为 0。

5.5.3 两种实现

```
# 方式一：加法掩码（加到分数上）
def create_causal_mask(seq_len, device='cuda'):
    return torch.triu(
        torch.full((seq_len, seq_len), float('-inf'), device=device),
        diagonal=1,
    )

scores = scores + create_causal_mask(seq_len, scores.device)
attn = torch.softmax(scores, dim=-1)

# 方式二：布尔掩码（masked_fill）
mask = torch.tril(torch.ones(seq_len, seq_len, dtype=torch.bool))
scores = scores.masked_fill(~mask, float('-inf'))
```

5.5.4 工程意义：跳过近一半计算

原文的关键一句：**高性能实现（如 FlashAttention）不显式构造 N×N 矩阵再相加，而是在 tiled 计算中通过索引判断跳过上三角区域**，既省显存又减少无效计算。FlashAttention-2 正是利用这一点，**在处理上三角 tile 时直接跳过，减少了接近一半的计算量**。

【推断】所以"因果掩码"不只是正确性需求，它还是一个免费的 2 倍加速机会——只要你的 kernel 实现知道自己在处理 causal 场景。这也是为什么很多 Attention kernel 要分 `causal=True/False` 两个版本。

5.6 $O(N^2)$ 到底来自哪里：逐项拆 FLOPs

原文把 Self-Attention 的六次矩阵乘全部列出（N 为序列长度，d 为 model 维度）：

运算	形状	FLOPs
Q = X·W_Q	(N,d)×(d,d)	$2Nd^2$
K = X·W_K	(N,d)×(d,d)	$2Nd^2$
V = X·W_V	(N,d)×(d,d)	$2Nd^2$
S = Q·K^T	(N,d)×(d,N)	$2N^2d$
O = S·V	(N,N)×(N,d)	$2N^2d$
Final = O·W_O	(N,d)×(d,d)	$2Nd^2$
总计		**$8Nd^2 + 4N^2d$**

原文的判据非常清晰：

条件	主导项	瓶颈在哪
N < d（短序列）	$8Nd^2$	QKV 投影的 GEMM
N > d（长序列）	$4N^2d$	Attention 矩阵计算

所以"$O(N^2)$"不是一句口号，它精确地等于 `4N²d` 这一项。 当 S 从 4K 涨到 128K，这一项涨 1024 倍。

5.7 计算瓶颈：Compute Bound 还是 Memory Bound

原文用算术强度（Arithmetic Intensity = FLOPs / Bytes）判断，并给了 A100 的具体比值：

```
A100: FP16 算力 ≈ 312 TFLOPS，HBM 带宽 ≈ 2 TB/s
算力/带宽比 ≈ 156 FLOP/Byte
```

阶段	矩阵形状	算术强度	结论
Prefill（处理完整 prompt）	QKV 投影是大矩阵乘，seq_len 维很大	高	Compute Bound，GPU 算力满载
Decode（逐 token 生成）	每步只有 1 个 token，投影退化为 GEMV	极低	Memory Bound，时间花在搬权重和 KV Cache

5.8 O(N^2) 的显存瓶颈：一个让人记住的数字

标准实现要显式存储完整 N×N 注意力矩阵。原文给的两个算例：

```
seq_len = 128K, num_heads = 32, FP16:
32 × 128K × 128K × 2B = 32 × 16384M × 2 ≈ 1,048,576 MB = 1 TB
```

```
seq_len = 8K, num_heads = 32, FP16:
32 × 8K × 8K × 2B = 4 GB
```

原文的结论：**即使序列长度"只有" 8K，注意力矩阵也需要 4 GB，已经是一个不可忽视的开销。** 这个 O(N^2) 的显存瓶颈，正是 FlashAttention 要解决的核心问题。

5.9 FlashAttention 的核心思想（只讲与本章相关的部分）

FlashAttention 不改变计算结果（**是精确计算，不是近似**），只重新编排计算顺序。

5.9.1 tiling：不物化 N×N

```
标准实现：
  Q, K 全部加载 → 计算完整 N×N 矩阵 → 写回 HBM
  → 读取做 Softmax → 写回 HBM → ...

FlashAttention：
  将 Q 分成 T_r 块，K/V 分成 T_c 块
  for 每一块 Q_i:
      for 每一块 K_j, V_j:
          从 HBM 加载 Q_i, K_j, V_j 到 SRAM
          在 SRAM 内算 Q_i @ K_j^T → 局部 softmax → 乘 V_j
          累积到 Q_i 的输出
      将 Q_i 的最终输出写回 HBM
```

注意力矩阵的这一小块始终驻留在 SRAM 中，永远不需要把完整的 N×N 矩阵写入 HBM。

5.9.2 online softmax 让分块在数学上成立

分块带来一个矛盾：Softmax 需要对每行所有元素归一化（要全局最大值和全局求和），但分块时每次只看到一部分。

解法就是第 3 章 3.7.2 的递推式——新块到来时动态更新全局最大值与分母，并修正之前块的结果：

```
m_new = max(m1, m2)
l_new = l1·exp(m1 − m_new) + l2·exp(m2 − m_new)
O_new = [O1·l1·exp(m1−m_new) + O2_local·l2·exp(m2−m_new)] / l_new
```

不断累积直到所有 K 块处理完，**最终结果与标准 Attention 的全局 Softmax 在数学上完全一致**。

5.9.3 省下的是什么：HBM 访问量

标准 Attention 的 HBM 访问（每一项都是 $O(N^2)$）：

```
写入 S (N×N)          → O(N²) 写
读取 S 做 Softmax     → O(N²) 读
写入 P = softmax(S)   → O(N²) 写
读取 P 做 PV          → O(N²) 读
```

FlashAttention 把这些全部消掉，代价是**反向时要重算**（用计算换 IO）。原文给的定量结论（学习路线的检验标准里）：**额外显存从 $O(N^2)$ 降到 $O(N)$，HBM 访问量从 $\Theta(N^2)$ 降到 $\Theta(N^2d^2/M)$**（M 为 SRAM 大小），仍随 N^2 增长但常数大幅减小。

第 6 章 从 MHA 到 MQA / GQA / MLA：KV Cache 驱动的架构演进

6.1 MHA：把 d_model 切成 N_h 个子空间

多头注意力的动机不是"为了更快"，而是**为了让不同的头学习不同的关注模式**。原文的工程视角表述是：把 `d_model` 切分为 `num_heads` 个子空间，每个头独立做一次完整的 Attention。

```
X: (B, S, d_model)
  ├─ W_Q → (B, S, N_h·D_h) → view → (B, N_h, S, D_h)
  ├─ W_K → (B, S, N_h·D_h) → view → (B, N_h, S, D_h)
  └─ W_V → (B, S, N_h·D_h) → view → (B, N_h, S, D_h)

每个头: softmax(Q_h K_hᵀ/√D_h) V_h : (B, N_h, S, D_h)
拼接:   (B, S, N_h·D_h) = (B, S, d_model)
输出:   × W_O → (B, S, d_model)
```

`d_model = N_h × D_h`。约束是 `d_model % N_h == 0`。

6.2 关键洞察：参数量 vs KV Cache 是两笔账

这是本章最重要的一句话：**MHA → GQA 的演进，优化的是推理时的 KV Cache，不是训练参数量。**

先看参数量（以 LLaMA-2-7B 配置 `d=4096, h=32, d_k=128` 为例）：

```
W_Q, W_K, W_V, W_O 各 (4096, 4096) → 4 × 4096² = 67,108,864 ≈ 67M
```

再看 KV Cache（公式见第 11 章）：

```
单 token KV Cache = 2 × L × h_kv × d_k × bytes
```

把 Q 头数和 KV 头数分开（`h_q` 个 Q 头，`h_kv` 个 KV 头），三种方案的差异就出来了。原文给的对比表（d=4096、d_k=128、h_q=32、L=32、FP16）：

指标	MHA (h_kv=32)	GQA (g=8, h_kv=8)	MQA (h_kv=1)
W_K 参数量	32×128×4096 = 16M	8×128×4096 = 4M	1×128×4096 = 0.5M
W_V 参数量	16M	4M	0.5M
单 token KV Cache	2×32×128×2B = 16 KB	2×8×128×2B = 4 KB	2×1×128×2B = 0.5 KB
4096 token / 层	64 MB	16 MB	2 MB
模型质量	最好	接近 MHA	有下降

原文的结论：**从 MHA 到 GQA(g=8)，KV Cache 缩小为 1/4，参数量减少，但模型质量几乎不受影响。这就是 GQA 成为主流选择的原因。**

【推断】注意这张表的"模型质量"列是从经验结论来的定性描述（原文也是定性），不要把它当成严格测量值。MQA 的质量下降在长上下文场景更明显，这也是为什么后来的主流模型都停在 GQA 而不是继续退到 MQA。

6.3 三种方案的机制差异

方案	机制	KV 头数	通信/切分影响
MHA	每个 Q 头有独立的 K/V 头	h_kv = h_q	TP 切分时 KV 头也要切，切完每卡仍持有 h_kv/TP 个
MQA	所有 Q 头共享一组 K/V	h_kv = 1	极致压缩，但表达能力受限
GQA	每组 g 个 Q 头共享一组 K/V	h_kv = h_q / g	主流折中，TP 切分需保证 h_kv 能被 TP 整除

原文强调的工程约束：**TP 切分时 `h_kv` 必须能被 TP 度整除**。LLaMA-2-70B 用 8 个 KV 头服务 64 个 Q 头，就限制了它的 TP 只能取 1/2/4/8。

6.4 MLA：换一个方向压缩

GQA 的思路是"减少 KV 头的数量"，MLA（Multi-head Latent Attention，DeepSeek-V2）的思路是"**降低每个表示的维度**"。

```
标准 MHA: X → W_K → K          (缓存 K, 维度 = h_kv × d_k)
MLA:      X → W_DKV → c        (缓存低维 latent c), 推理时 c → W_UK → K 按需解压
```

原文给出的关键数据：**MLA 通过低秩压缩将 KV Cache 大幅缩小（DeepSeek-V2 报告约 93.3% 的压缩率），同时通过精心设计的上投影矩阵保持了模型质量。**

方案	压缩对象	代价
GQA	KV 头数	少量质量
MLA	每个表示的维度（低秩）	实现复杂度：多一组上下投影矩阵，且与 RoPE 的结合需要特殊处理

第 7 章 FFN 与 MoE：参数量的 2/3 在这里

7.1 FFN 的角色：token 内的运算

原文给了一个精准的二分：

模块	运算类型	职责
Self-Attention	**token 间**的运算，输出依赖序列中所有 token 的组合	信息交互、原料分拣
FFN	**token 内**的运算，对每个 token 独立施加相同非线性变换	深加工、记忆知识

为什么必须有 FFN？原文的论证链是：Attention 本质是加权求和（线性运算），仅靠它表达能力受限；FFN 引入非线性，按万能近似定理（Universal Approximation Theorem），带非线性激活的两层前馈网络可以逼近任意连续函数。

近年研究还发现 FFN 在大模型里扮演"**知识库**"角色——大量事实性知识编码在 FFN 参数中，Attention 更多负责组织和提取。**这也解释了为什么 FFN 占了模型大部分参数：它需要足够存储容量来记忆海量知识。**

7.2 "展开-压缩"结构与四条理由

标准 FFN（两层线性 + 激活）：

```
FFN(x) = σ(x·W_1 + b_1)·W_2 + b_2
W_1: (d_model, d_ff)     升维
W_2: (d_ff, d_model)     降维
```

以 `d_model=4096, d_ff=4×4096=16384` 为例：

阶段	维度
输入	(N, 4096)
升维	(N, 16384)
激活	(N, 16384)

阶段	维度
降维	(N, 4096)

为什么先升维再降维？原文给了四条：

1. **高维空间线性可分性更好**——一维混在一起的红蓝球，抛到三维更容易用平面分开；
2. **激活在高维更有效**——每个维度是一个特征检测器，维度越高能同时检测的模式越多；
3. **降维是信息蒸馏**——迫使模型只保留最重要的信息；
4. **与 Autoencoder 相反**——Autoencoder 先压再还原（学低维表示），FFN 先展开再压缩（在高维做非线性变换后提纯）。

7.3 激活函数演进：ReLU → GELU → Swish → SwiGLU

激活	公式	优点	缺点
ReLU	`max(0, x)`	最简单、最快	神经元死亡（负区间梯度恒 0）
GELU	`x·Φ(x)`，常用 tanh 近似	处处光滑可微、软门控	计算量比 ReLU 大（erf/tanh）
Swish	`x·σ(βx)`	自门控、光滑，深层网络优于 ReLU	多一次 sigmoid
SwiGLU	`(Swish(xW_gate) ⊗ xW_up)W_down`	门控选择性、实验最佳	3 次 GEMM 而非 2 次

ReLU 的"神经元死亡"值得展开：某个神经元输入恒为负时输出为 0、梯度也为 0，权重得不到更新，下一轮输入大概率仍为负——**永久性死亡**。原文提到实际训练中有时可观察到超过 40% 的 ReLU 神经元处于死亡状态。

GELU 的核心思想是把激活看作**概率性门控**：以概率 Φ(x) 保留 x，以概率 1−Φ(x) 置零，取期望。

7.4 SwiGLU：为什么是 8/3 倍

SwiGLU 完整形式（三个权重矩阵）：

```
FFN_SwiGLU(x) = [Swish(x·W_gate) ⊗ (x·W_up)]·W_down
```

如果继续用 `d_ff = 4×d_model`，总参数会变成 `3 × d_model × 4d_model = 12d_model²`，比标准 FFN 的 `8d_model²` 多 50%。为了保持总参数量不变：

```
3 × d_model × d_ff = 8 × d_model²
⇒ d_ff = (8/3) × d_model ≈ 2.667 × d_model
```

原文的工程细节：`(8/3) × 4096 = 10922.67`，**取整到 11008（256 的倍数，有利于 GPU 计算对齐）**——这就是 LLaMA 配置里 `d_ff=11008` 的由来。

对比表（`d_model=4096`）：

对比维度	标准 FFN	SwiGLU FFN
权重矩阵数量	2	3
中间维度	4×d_model	(8/3)×d_model
总参数量	$8d_model^2$	$8d_model^2$
d=4096 时	134M	135M
前向 GEMM 次数	2	3
激活施加方式	全局	门控选择性

关键发现：两种结构总参数量几乎相同，但 SwiGLU 通过门控实现更好效果，代价是前向多一次 GEMM。

7.5 参数量分布：FFN 占 2/3

`d_model=4096, d_ff=11008` 的单个 Block：

模块	参数量	占 Block 比例
Self-Attention（W_Q,W_K,W_V,W_O）	$4 \times 4096^2 = 67{,}108{,}864$	~33%
FFN（SwiGLU）	$3 \times 4096 \times 11008 = 135{,}266{,}304$	~67%
RMSNorm × 2	$2 \times 4096 = 8{,}192$	<0.01%
单 Block 合计	**202,383,360（≈202M）**	100%

原文的工程含义：**FFN 参数量约为 Attention 的 2 倍。任何针对 FFN 的优化（并行切分、量化、专家化）都会对整体效率产生显著影响。**

7.6 张量并行切分 FFN：Megatron 的列切 + 行切

这是"从模型结构直接读出并行方案"的最好例子。

7.6.1 切分方案

```
第一步：W_up（和 W_gate）按列切分
  GPU0: W_up[:, :d_ff/2]   GPU1: W_up[:, d_ff/2:]
  各自算 h_0 = act(x @ W_up_0)  h_1 = act(x @ W_up_1)    形状 (N, d_ff/2)

第二步：W_down 按行切分
  GPU0: W_down[:d_ff/2, :]  GPU1: W_down[d_ff/2:, :]
  各自算 out_0 = h_0 @ W_down_0   out_1 = h_1 @ W_down_1  形状 (N, d_model)

第三步：AllReduce 求和
  output = out_0 + out_1
```

数学依据（行向量视角）：

```
h·W_down = [h_0  h_1] · [W_down,0 ; W_down,1] = h_0·W_down,0 + h_1·W_down,1
```

7.6.2 为什么列切分能"免费"施加激活

原文的关键洞察：**激活函数是逐元素操作，不依赖其他维度的值，所以对完整向量施加激活等价于对各部分分别施加激活。**

7.6.3 通信开销：每 Block 两次 AllReduce

原文的结论：**整个 FFN 前向只需要一次 AllReduce**，每卡上的三次 GEMM 和逐元素操作都是本地计算。加上 Attention 的一次，**每个 Block 共两次 AllReduce**。

这也回答了第 15 章要展开的一个问题：为什么 TP 不能跨机——每层两次 AllReduce 乘以 80 层就是 160 次，再乘以前向反向。

7.6.4 工程约束：d_ff 必须能被 TP 整除

```
d_ff = 11008
TP=2 → 每卡 5504
TP=4 → 每卡 2752
TP=8 → 每卡 1376     都能整除
```

原文指出这也是 `d_ff` 优先取 2 的幂或 128/256 倍数的原因之一。

7.7 MoE：把 FFN 拆成多个专家

7.7.1 核心思想

与其用一个巨大 FFN，不如拆成多个较小的"专家"（Expert），每个专家是一个独立 FFN；**每个 token 只激活其中少数几个**。

```
非 MoE:   token ──→ [ 一个巨大 FFN ]                    全部参数都参与
MoE:      token ──→ Router ──→ [Expert 3] [Expert 17]   只算 Top-k 个
                               (其余专家休眠)
```

原文的例子：64 个专家、每 token 选 2 个（Top-2）：

- 模型总参数量 = 64 个专家之和（很大）；
- 每 token 计算量 = 2 个专家的计算量（很小）。

这实现了"参数量大但计算量小"——有足够容量存知识，但训练/推理计算成本可控。

7.7.2 对 Infra 的三个新问题

【补充】原文在路线图里点明了 MoE 给工程带来的挑战，这里按机制展开：

问题	机制	影响
Expert Parallelism（EP）	不同专家放在不同卡上	引入 all-to-all 通信（第 15 章），比 AllReduce 更难优化
负载均衡	Router 可能把 token 集中到少数专家	需要 aux loss 或 capacity factor 限制；热点专家成为瓶颈
显存与带宽	总参数大但激活少	权重显存按总参数算，带宽按激活算，二者不再同比例

第 8 章 位置编码：Sinusoidal 与 RoPE

8.1 为什么必须注入位置：Attention 是排列等变的

原文给了一串语序敏感的例子（"他借给我一本书" vs "我借给他一本书"），然后做了严格证明。

设 Π 是一个 N×N 排列矩阵（每行每列恰好一个 1），`ΠX` 表示把 X 的行打乱重排，则：

$$
\begin{aligned}
&Q' = \Pi X W_Q = \Pi Q, \quad K' = \Pi K, \quad V' = \Pi V \\
&\text{softmax}(Q'K'^{\top}/\sqrt{d_k})\cdot V' \\
&\quad = \text{softmax}(\Pi Q K^{\top} \Pi^{\top}/\sqrt{d_k})\cdot \Pi V \\
&\quad = \Pi\cdot\text{softmax}(QK^{\top}/\sqrt{d_k})\cdot \Pi^{\top}\cdot \Pi V \\
&\quad = \Pi\cdot\text{softmax}(QK^{\top}/\sqrt{d_k})\cdot V
\end{aligned}
$$

这就是排列等变性（permutation equivariance）：输入打乱后，输出只是做了同样的打乱，每个 token 聚合到的信息内容完全不变。

原文的结论一句话：**原始 Attention 把输入当作一个集合而非序列处理——位置 0 的 token 和位置 99 的 token 在它眼中没有任何区别。**

8.2 绝对 vs 相对：两种设计思路

维度	绝对位置编码	相对位置编码
信息注入点	输入层（加到 embedding 上）	Attention 计算过程中
直觉	给每个人发门牌号，靠差值推距离	交谈时直接告诉你们隔了几间房
持久性	随深度增加可能被稀释	每层 Attention 都重新注入
外推能力	通常较弱	通常较强
长度泛化	难处理训练未见长度	相对容易扩展
代表	Sinusoidal、可学习编码	RoPE、ALiBi

当前大模型几乎全部采用相对位置编码（以 RoPE 为主）。

8.3 Sinusoidal 编码

8.3.1 公式

```
PE(pos, 2i)   = sin(pos / 10000^(2i/d))
PE(pos, 2i+1) = cos(pos / 10000^(2i/d))
```

`pos` 是位置（从 0 开始），`i` 是维度对索引（0 ≤ i < d/2）。每一对相邻维度 `(2i, 2i+1)` 共享同一个频率 `ω_i = 1/10000^(2i/d)`；`i` 从 0 增大到 d/2−1 时，频率从 1 指数衰减到 1/10000。

8.3.2 收音机调谐的直觉

原文这个比喻很好用：

> 一排收音机旋钮，最左边的调到极高频率（移动一个位置几乎转一整圈），最右边调到极低频率（走过一百个位置才转一小角度）。给你位置编号 42，让每个旋钮转到对应角度——所有指针组成的图案对 42 是唯一的。换成 43，所有指针都有不同程度的偏转，形成另一个唯一图案。

- 高频维度（小 i）变化快 → 能精确区分相邻位置；
- 低频维度（大 i）变化慢 → 能区分相距遥远的位置；
- **所有频率联合起来，为每个位置提供唯一的"频谱指纹"。**

8.3.3 实现

```
import numpy as np

def sinusoidal_position_encoding(max_len, d_model):
    PE = np.zeros((max_len, d_model))
    position = np.arange(max_len)[:, np.newaxis]              # (max_len, 1)
    dim_pair_index = np.arange(0, d_model, 2)[np.newaxis, :]   # (1, d_model/2)
    div_term = np.exp(dim_pair_index * -(np.log(10000.0) / d_model))
    PE[:, 0::2] = np.sin(position * div_term)                 # 偶数维 sin
    PE[:, 1::2] = np.cos(position * div_term)                 # 奇数维 cos
    return PE
```

Sinusoidal 没有任何可学习参数，这是它与可学习编码的本质区别。

8.4 可学习位置编码

给每个位置一个可学习向量，`W_pos ∈ R^{max_len × d}`，加到 embedding 上。BERT/GPT-2 用过。

优点	缺点
完全由数据决定，不受固定公式限制	参数量随 max_len 增长
实现最简单	**超出训练长度就完全无能为力**（没有对应行）

8.5 RoPE：旋转位置编码

8.5.1 核心思想

RoPE 不是"加一个位置向量"，而是**把 Q 和 K 的向量按位置角度旋转**，使得旋转后的内积只依赖相对位置差。

把 `d_k` 维向量按两两一组看成 `d_k/2` 个二维平面，第 i 组用频率 `θ_i = 1/10000^(2i/d_k)`，位置 pos 的旋转角为 `pos·θ_i`：

```
[x1, x2] → [x1·cos(pos·θ) − x2·sin(pos·θ),
            x1·sin(pos·θ) + x2·cos(pos·θ)]
```

8.5.2 为什么旋转能编码相对位置

【补充】两个旋转后的向量做内积，其结果只与 `(pos_q − pos_k)` 有关：

```
⟨R(pos_q)·q, R(pos_k)·k⟩ = ⟨q, R(pos_k − pos_q)·k⟩
```

这条恒等式是 RoPE 的全部价值所在：**注意力分数天然携带相对距离信息，不需要额外的相对位置偏置表。**

8.5.3 实现（来自 Decoder Block 章节的完整代码）

```
def precompute_rope_freqs(d_k: int, max_seq_len: int, theta: float = 10000.0):
    """预计算 RoPE 的旋转频率"""
    freqs = 1.0 / (theta ** (torch.arange(0, d_k, 2).float() / d_k))
    positions = torch.arange(max_seq_len).float()
    angles = torch.outer(positions, freqs)          # (max_seq_len, d_k//2)
    return torch.cos(angles), torch.sin(angles)     # 各 (max_seq_len, d_k//2)

def apply_rope(x, cos, sin):
    """x: (B, S, N_h, d_k); cos/sin: (S, d_k//2)"""
    seq_len = x.shape[1]
    cos = cos[:seq_len].unsqueeze(0).unsqueeze(2)   # (1, S, 1, d_k//2)
    sin = sin[:seq_len].unsqueeze(0).unsqueeze(2)
    x1 = x[..., : x.shape[-1] // 2]
    x2 = x[..., x.shape[-1] // 2 :]
    rotated = torch.cat([x1 * cos - x2 * sin, x1 * sin + x2 * cos], dim=-1)
    return rotated
```

> 实现细节：上面这种"前后两半配对"的写法（GPT-NeoX 风格）与"相邻两两配对"（原始论文风格）在数学上等价，只要 cos/sin 的频率排列与配对方式一致。**混用两种风格是最常见的 RoPE 实现 Bug。**

8.5.4 三条工程约束（原文重点）

约束	说明
RoPE 只作用于 Q 和 K	V 不需要旋转，因为 V 承载"内容信息"而非"位置匹配信号"
RoPE 不引入可学习参数	频率是预计算的，cos/sin 表可缓存
必须与 KV Cache 的写入位置对齐	增量解码时新 token 的位置索引必须正确，否则位置错乱且不报错

第 9 章 LayerNorm / RMSNorm 与 Pre-Norm vs Post-Norm

9.1 为什么深层网络难训练

原文从两个角度切入：

梯度消失与梯度爆炸：L 层网络的梯度是连续偏导相乘。若每层雅可比矩阵的谱范数为 σ，经过 L 层连乘后梯度以 `σ^L` 变化：σ<1 指数衰减，σ>1 指数增长，**只有 σ 恰好等于 1 才能稳定——而这在实际训练中几乎不可能自然满足。**

退化问题（Degradation Problem）：2015 年何恺明等人发现更深的网络反而更差，不是过拟合（训练误差也更高），而是优化本身出了问题。理论上 56 层不该比 20 层差（多出的层学成恒等映射即可），但**让网**

络"学会什么都不做"远比想象的困难。

9.2 残差连接：梯度的高速公路

```
output = x + F(x)
```

为什么学"残差"比学"完整映射"容易？原文的建筑设计类比很好：方式 A 从零画一份全新的终版图纸；方式 B 在初版图纸上用红笔标注修改意见。**方式 B 的认知负担小得多。**

更关键的是数学性质。有残差时：

```
x_{l+1} = x_l + F_l(x_l)
∂x_{l+1}/∂x_l = I + ∂F_l/∂x_l
```

展开从第 L 层到第 l 层的梯度，连乘中**必然包含一项恒为 I 的乘积**（所有括号都选 I 那一项）：

```
∂L/∂x_l = (∂L/∂x_L)·(I + 其他交叉项)
```

无论中间各层的 ∂F/∂x 有多小甚至趋近于零，梯度都能通过这条"恒等通道"原封不动传到第 l 层。 这就是"梯度高速公路"。

原文还有一个很精炼的总结：**残差连接把反向传播中的纯"乘法链"变成了"加法 + 乘法"的混合结构。** 纯乘法链对数值极其敏感（连续乘 0.9 五十次），加法的引入让梯度有了"保底值"。

9.3 归一化方法全景

方法	归一化维度	适合	为什么
BatchNorm	沿 batch 维，对每个特征	CV	NLP 的 batch 内序列长度不一，统计量不稳定
LayerNorm	沿特征维，对每个样本	NLP/Transformer	与 batch 无关，单样本即可算
RMSNorm	沿特征维，只算均方根	现代大模型	省去减均值，少一次归约

LayerNorm：

```
y = γ · (x − μ) / √(σ² + ε) + β
μ = mean(x, dim=−1),  σ² = var(x, dim=−1)
```

RMSNorm：

```
y = x / √(mean(x²) + ε) · γ
```

RMSNorm 只有一个可学习参数 γ，没有 β。 原文从 kernel 角度解释了它为什么流行：

> RMSNorm 去掉了均值计算，只需计算 Σx^2。这不仅减少计算量，更重要的是简化了 reduction 操作——只需要一个 reduction（sum of squares）而非两个（sum 和 sum of squares），kernel 实现更简单，数值稳定性问题也更少。

9.4 Pre-Norm vs Post-Norm

两种数据流（原文的两张图，这里用文字版）：

```
Post-Norm（原始论文 2017）：          Pre-Norm（GPT-2 起，现主流）：
  x ──┬──→ SubLayer ──→ Add ──→ LN      x ──┬──→ LN ──→ SubLayer ──→ Add
      │                  ↑                  │                       ↑
      └──────────────────┘                  └───────────────────────┘
  output = LN(x + SubLayer(x))           output = x + SubLayer(LN(x))
```

9.4.1 梯度流差异（本章最重要的一段）

Post-Norm 的梯度：

$$\partial L/\partial x_l = (\partial L/\partial x_L) \cdot \Pi_k \left[\partial LN_k/\partial(x_k+F_k) \cdot (I + \partial F_k/\partial x_k) \right]$$

LayerNorm 的雅可比矩阵不是单位阵——它涉及均值和方差的梯度，会对回传梯度做非平凡线性变换。**这些变换累乘起来可能导致方向与大小显著变化**；在 96 层的 GPT-3 上可能引发梯度爆炸，需要精心设计 warmup。

Pre-Norm 的梯度：

$$\partial L/\partial x_l = (\partial L/\partial x_L) \cdot \Pi_k \left[I + \partial(F_k(LN_k(x_k)))/\partial x_k \right]$$

每个因子都是"I + 某个东西"。展开后必然包含 I 项，**梯度可通过恒等路径直达任意浅层，不经过任何 LayerNorm**；LN 的雅可比只出现在交叉项里，不阻塞主通路。

一句话：Pre-Norm 让残差路径保持"纯净"。

9.4.2 实验证据与工程结论

结论	来源/依据
Pre-Norm 对学习率更鲁棒	Xiong et al. 2020 证明 Post-Norm 初始化时各层输出方差随深度线性增长，必须小学习率 + 长 warmup；Pre-Norm 方差天然稳定
Post-Norm 的理论优势难兑现	充分收敛时略好（原文引用的量级是约 0.1–0.5 BLEU），但需极精细调参
大规模训练零容忍不稳定	175B 训练可能花费数百万美元，一次发散就前功尽弃

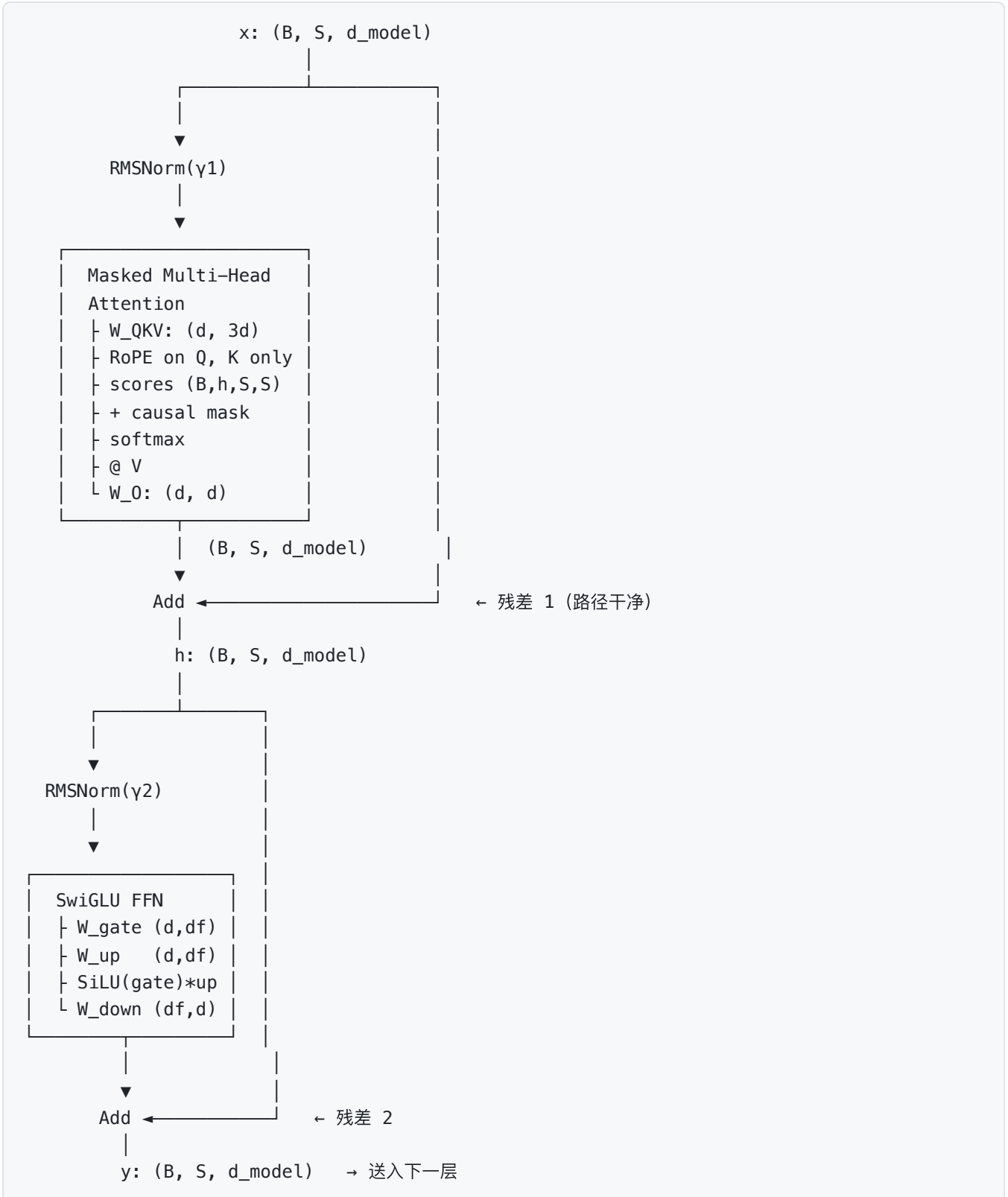
原文的推理链条值得记住：

```
大模型 → 层数多（64–128 层） → 梯度流路径长
→ 需要极稳定的梯度传播 → 残差路径必须干净
→ Pre-Norm 不在残差路径上放 LN → 梯度直通
→ 训练更稳定、超参更好调 → 工程友好 → 成为默认选择
```

第 10 章 Decoder Block 完整数据流、维度表与参数量手算

本章是 Transformer 部分的"落账"章：把前面所有零件装成一个 Block，然后把参数量、FLOPs、显存全部算清楚。**这是 AI Infra 工程师最核心的手工技能。**

10.1 一个 Pre-Norm Decoder-only Block 的数据流



原文特别标注了三个容易做错的细节：

细节	正确做法	常见错误
归一化选型	LLaMA 系列及后续多数用 RMSNorm	照抄原论文用 LayerNorm（在 LLaMA 结构上会多一组参数）

细节	正确做法	常见错误
RoPE 作用范围	只作用于 Q 和 K	也给 V 加旋转（破坏内容信息）
残差路径	两条都从 Add 节点直拉，中间无任何变换	在残差路径上插入 dropout/norm（破坏梯度直通）

10.2 完整维度表：从输入到 logits

以 `B=2, S=128, d_model=4096, N_h=32, D_h=128, d_ff=11008, V=32000` 为例：

步骤	操作	输入 shape	输出 shape	备注
1	token ids	—	(2, 128)	int64
2	Embedding 查表	(2, 128)	(2, 128, 4096)	参数量 V×d
3	RMSNorm	(2, 128, 4096)	(2, 128, 4096)	每 token 独立
4	QKV 投影	(2, 128, 4096)	(2, 128, 12288)	一次 GEMM 后 split
5	reshape + transpose	(2, 128, 3, 32, 128)	Q/K/V: (2, 32, 128, 128)	head 维提前
6	RoPE	(2, 32, 128, 128)	同 shape	只对 Q、K
7	QK^T	(2,32,128,128)×(2,32,128,128)^T	(2, 32, 128, 128)	**$O(S^2)$ 出现点**
8	scale + mask	(2, 32, 128, 128)	同 shape	/√128
9	softmax(dim=−1)	(2, 32, 128, 128)	同 shape	每行和 1
10	@ V	(2,32,128,128)×(2,32,128,128)	(2, 32, 128, 128)	
11	transpose + merge	(2, 32, 128, 128)	(2, 128, 4096)	需 contiguous
12	W_O	(2, 128, 4096)	(2, 128, 4096)	
13	Add（残差1）	—	(2, 128, 4096)	
14	RMSNorm	(2, 128, 4096)	(2, 128, 4096)	
15	W_gate / W_up	(2, 128, 4096)	各 (2, 128, 11008)	两次 GEMM
16	SiLU(gate) * up	两个 (2,128,11008)	(2, 128, 11008)	逐元素
17	W_down	(2, 128, 11008)	(2, 128, 4096)	
18	Add（残差2）	—	(2, 128, 4096)	一个 Block 输出
19	× L 层	(2, 128, 4096)	(2, 128, 4096)	L=32
20	最终 RMSNorm	(2, 128, 4096)	(2, 128, 4096)	

步骤	操作	输入 shape	输出 shape	备注
21	LM Head	(2, 128, 4096)	(2, 128, 32000)	参数量 d×V
22	取最后位置 + softmax	(2, 32000)	(2, 32000)	只在生成时取最后

这张表就是"Transformer 白板默写"的评分标准——能不看资料写出来，第零层就过关了。

10.3 主流模型维度配置对比

原文的配置表（这张表决定了后面所有算例）：

模型	d_model	h (Q 头)	h_kv	d_k	L	d_ff	V	max_seq_len
LLaMA-2-7B	4096	32	32 (MHA)	128	32	11008	32000	4096
LLaMA-2-13B	5120	40	40 (MHA)	128	40	13824	32000	4096
LLaMA-2-70B	8192	64	8 (GQA)	128	80	28672	32000	4096
GPT-3 175B	12288	96	96 (MHA)	128	96	49152	50257	2048
Mistral-7B	4096	32	8 (GQA)	128	32	14336	32000	32768

原文提炼的三条规律：

1. **d_k 基本恒定为 128**。模型变大时增加的是头数 h 和层数 L，而非单头维度——因为 128 维已足以让每个头捕捉一种有意义的注意力模式。
2. **d_ff 与 d_model 的比例**：标准 FFN 是 4×；SwiGLU 用 (8/3)× 再向上取整到方便的值（LLaMA-2-7B：10922.67 → 11008）。
3. **GQA 的引入**：LLaMA-2-70B 用 8 个 KV 头服务 64 个 Q 头，KV Cache 降到 MHA 的 1/8。

10.4 参数量手算：LLaMA-2-7B 全流程

10.4.1 公式先立好

```
Attention (MHA) :  4·d²                      (W_Q,W_K,W_V,W_O 各 d²)
Attention (GQA) :  d² + 2·d·h_kv·d_k + d²    (Q 全宽, K/V 只有 h_kv 组)
SwiGLU FFN:         3·d·d_ff
RMSNorm (每 Block) : 2·d
单 Block 合计:     4d² + 3·d·d_ff + 2d
模型总计:          L × 单Block + V·d (Embedding)  + d (最终 norm)  + d·V (LM Head)
```

10.4.2 逐项代入（d=4096, h=32, d_k=128, d_ff=11008, L=32, V=32000）

组件	计算	参数量
Attention（单 Block）	4×4096^2	67,108,864
FFN（单 Block）	$3 \times 4096 \times 11008$	135,266,304
RMSNorm（单 Block）	2×4096	8,192
单 Block 合计		**202,383,360（≈202M）**
Token Embedding	32000×4096	131,072,000
32 层 Block	$32 \times 202,383,360$	6,476,267,520
最终 RMSNorm	4096	4,096
LM Head	4096×32000	131,072,000
总计		**6,738,415,616（≈6.74B）**

原文的重要提醒：**LLaMA-2 的 Embedding 和 LM Head 是两套独立权重（不共享），所以 6.74B 已包含两份 131M；若共享（Weight Tying）则需减去 131M，约 6.61B。官方标注的 "7B" 是近似值。**

10.4.3 校验：13B 也要能算

配置 `d=5120, h=40, d_k=128, d_ff=13824, L=40, V=32000`：

组件	计算	参数量
Attention	4×5120^2	104,857,600
FFN	$3 \times 5120 \times 13824$	212,336,640
RMSNorm	2×5120	10,240
单 Block 合计		317,204,480
Embedding	32000×5120	163,840,000
40 层	$40 \times 317,204,480$	12,688,179,200
LM Head	5120×32000	163,840,000
最终 norm		5,120
总计		**13,015,864,320（≈13.0B）**

与官方标注的 13B 吻合。 这条校验很重要：它证明你的公式是对的，而不是碰巧凑上。

10.5 FLOPs 估算：2N、6N 与 Attention 修正项

10.5.1 三条公式

```
矩阵乘 (M,K)×(K,N):  FLOPs ≈ 2·M·K·N      (乘加各算一次)

前向 (per token) :     FLOPs_forward ≈ 2N
训练 (per token) :     FLOPs_train   ≈ 6N      (反向约为前向 2 倍)

完整前向 (含 Attention 的 S² 项) :
  FLOPs_forward ≈ 2Ns + 4·L·h·s²·d_k
```

原文的推导：线性层部分每个参数在前向中恰好参与一次矩阵乘，贡献约 2 FLOPs，乘序列长度 s；Attention 部分每层每个头有两个 `(s,d_k)×(d_k,s)` 矩阵乘，各贡献 $2s^2d_k$，共 $4s^2d_k$。

什么时候 2Ns 够用？ 原文给的判据：s 较短（如 2048）时第二项远小于第一项，"2Ns" 是好近似；**当 s 很长（如 128K）时，Attention 计算量可能接近甚至超过线性层。**

10.5.2 完整训练算例（这段值得背）

LLaMA-2-7B 用 2 万亿 token 训练：

```
FLOPs = 6 × 6.7e9 × 2e12 = 8.04e22
```

1000 张 A100（BF16 峰值 312 TFLOPS/张，假设 MFU=50%）：

```
有效算力 = 1000 × 312e12 × 0.5 = 1.56e17 FLOPS
训练时间 = 8.04e22 / 1.56e17 = 515,385 秒 ≈ 6 天
```

原文的评注：**这与实际公开的训练时间量级相符。** 这类"量级对得上"的校验，是判断自己算没算错的最好方法。

10.5.3 推理的"反直觉"算例

LLaMA-2-7B 在 A100-80GB 上单 token Decode：

```
FLOPs = 2 × 6.7e9 = 13.4 GFLOPs
理论耗时 = 13.4e9 / 312e12 = 0.043 ms
实际耗时 ≈ 10-20 ms          ← 差了几百倍
```

原文的解释（这是 Memory Wall 的最好例证）：**Decode 是 Memory Bound，需要从 HBM 搬运全部权重（13.4 GB FP16），A100 带宽 2 TB/s，光搬权重就要 13.4/2000 ≈ 6.7 ms**，再加 KV Cache 搬运和其他开销，10–20 ms 就合理了。

10.6 显存规划：三个公式打天下

10.6.1 权重显存

```
显存 = 参数量 × 每参数字节数
```

精度	每参数	7B	13B	70B
FP32	4 B	26.8 GB	52.0 GB	280.0 GB
FP16/BF16	2 B	13.4 GB	26.0 GB	140.0 GB
INT8	1 B	6.7 GB	13.0 GB	70.0 GB
INT4	0.5 B	3.35 GB	6.5 GB	35.0 GB

10.6.2 训练态四大组成

组成	每参数	7B（6.7B 参数）	说明
BF16 权重	2 B	13.4 GB	前向反向使用
BF16 梯度	2 B	13.4 GB	与权重同 shape
FP32 master weights	4 B	26.8 GB	BF16 精度不够表示微小更新
Adam 一阶动量 m	4 B	26.8 GB	梯度指数移动平均
Adam 二阶动量 v	4 B	26.8 GB	梯度平方指数移动平均
静态合计	**16 B**	**107.2 GB**	
Activation（估）	—	~25.8 GB	batch=4, seq=2048
总计		**~133 GB**	超过单张 A100-80GB

为什么需要 FP32 master weights? 原文的解释非常具体：BF16 只有约 3–4 位有效数字，更新权重时若学习率×梯度的值很小（如 1e-5），BF16 的精度不够表示这个微小增量，**更新会被"四舍五入"掉**；FP32 有约 7 位有效数字，能捕捉这些微小更新。

Activation 的估算公式：

```
M_act ≈ s × b × d × L × k        k ≈ 10–14（取决于是否用 Activation Checkpointing）
LLaMA-2-7B (s=2048, b=4, d=4096, L=32, k=12, FP16):
2048 × 4 × 4096 × 32 × 12 × 2 B ≈ 25.8 GB
```

10.6.3 推理态：权重 + KV Cache

```
M_kv = 2 × L × h_kv × d_k × s × b × bytes_per_element
```

LLaMA-2-7B（MHA，h_kv=32）：

```
单 token: 2 × 32 × 32 × 128 × 2 B = 524,288 B = 512 KB
seq=4096:  4096 × 512 KB = 2 GB
batch=16:  16 × 2 GB = 32 GB
```

组件	LLaMA-2-7B (FP16)	LLaMA-2-7B (INT4)
模型权重	13.4 GB	3.35 GB
KV Cache (seq=4096, batch=16)	32 GB	32 GB（KV 通常仍 FP16）
其他开销	~1 GB	~1 GB
总计	**~46.4 GB**	**~36.4 GB**

原文的提醒：**即使模型权重量化到 INT4，KV Cache 通常仍用 FP16**，因为量化 KV 对精度影响较大；KIVI/KVQuant 等研究可压到 INT4/INT2，但需额外校准与精度评估。

10.6.4 完整规划案例：13B 单卡能装多少 batch

场景：单张 A100-80GB 部署 LLaMA-2-13B 推理，目标 batch=8，seq=4096。

```
① 权重:    13B × 2 B = 26 GB
② KV:      L=40, h_kv=40, d_k=128
           单 token = 2×40×40×128×2 = 819,200 B = 800 KB
           seq=4096, batch=8: 4096 × 800 KB × 8 = 25.6 GB
③ 框架开销: ~2 GB
合计: 26 + 25.6 + 2 = 53.6 GB  < 80 GB  ✓ 余约 26 GB
```

若 batch 提到 16：KV 翻倍到 51.2 GB，合计 79.2 GB——**非常接近 80 GB 上限，实际运行大概率 OOM**。此时的选择（原文列的四条）：INT8 量化权重（13 GB，总计 66.2 GB）、改用 GQA 模型、KV Cache 量化、减少最大序列长度。

原文的总结：**"不是拍脑袋说'应该能跑'，而是精确地算出每一项开销，找到瓶颈，选择合适的优化策略。"**

第 11 章 自回归生成：Prefill / Decode 两阶段与 KV Cache

11.1 语言模型的本质：条件概率连乘

```
P(x_t | x_1..x_{t-1})
整句联合概率：P(x_1..x_T) = Π_t P(x_t | x_1..x_{t-1})
```

"填字游戏"的类比：每一步只能看到前面已填的字，从整个词表（32000～128000 个 token）中选最合理的。

11.2 从 hidden state 到概率：LM Head + Softmax

```
# hidden_state: (N, d_model) = (N, 4096)
# lm_head.weight: (vocab_size, d_model) = (32000, 4096)
logits = hidden_state @ lm_head.weight.T     # (N, 32000)
probs  = softmax(logits[-1])                 # 只取最后一个位置 → (32000,)
```

为什么只取最后一个位置：自回归模型只关心"下一个词"，而最后一个 token 位置的输出就包含了对下一个 token 的预测。

11.3 朴素生成循环与它的致命效率问题

```
def autoregressive_generate(model, prompt_tokens, max_new_tokens, temperature=1.0):
    generated = list(prompt_tokens)
    for _ in range(max_new_tokens):
        input_ids = torch.tensor([generated])       # (1, current_len)
        logits = model(input_ids)                   # (1, current_len, V)
        next_logits = logits[0, -1, :]              # (V,)
        probs = softmax(next_logits / temperature)
        next_token = torch.multinomial(probs, num_samples=1).item()
        generated.append(next_token)
    return generated
```

原文的评注：**每一步都把完整序列送入模型重新计算。当序列越来越长时，重复计算量急剧增加。** 生成第 t 个 token 时，前面 t−1 个 token 的 K/V 与之前完全相同——**这正是 KV Cache 要解决的问题。**

11.4 KV Cache：从 $O(S^2)$ 到 $O(S)$

11.4.1 机制

每层只需缓存已生成 token 的 K 和 V；新 token 到来时，只算它自己的 Q/K/V，把新 K/V 追加到缓存，然后用新 Q 与**全部历史 K** 做 Attention。

```
无 Cache：每步重算 [t 个 token] × L 层        总计算量 O(S²·L)  量级
有 Cache：每步只算 1 个 token + 读缓存        总计算量 O(S·L)   量级
```

11.4.2 显存代价（这是"显存刺客"）

```
M_kv = 2 × L × h_kv × d_k × s × b × bytes
       ↑   ↑    ↑     ↑    ↑   ↑
      K和V 层数 KV头数 头维 序列长 batch
```

模型	配置	单 token KV	seq=4096, batch=16
LLaMA-2-7B (MHA)	L=32, h_kv=32, d_k=128	512 KB	32 GB
LLaMA-2-13B (MHA)	L=40, h_kv=40, d_k=128	800 KB	51.2 GB
LLaMA-2-70B (GQA)	L=80, h_kv=8, d_k=128	320 KB	20.5 GB

注意 70B 的 KV Cache 反而比 13B 小——这就是 GQA 的威力（第 6 章）。

11.5 Prefill vs Decode：两个阶段的物理本质不同

维度	Prefill	Decode
处理对象	整个 prompt（一次 S 个 token）	每步 1 个 token
矩阵形状	大矩阵乘（S 维很大）	退化为 GEMV
算术强度	高	极低
瓶颈	**Compute Bound**	**Memory Bound**
关键指标	TTFT（首 token 延迟）	TPOT（每 token 延迟）
优化方向	并行切分、FlashAttention、Chunked Prefill	KV Cache 管理、批处理、投机解码

原文的经典问答：**"为什么 Prefill 快但 Decode 慢？"**——Prefill 一次处理整个 prompt，矩阵大、算力利用率高；Decode 每步只生成一个 token，矩阵退化成向量，大部分时间在等显存搬运 KV Cache。

11.7 采样策略

策略	规则	特点
Greedy	取 argmax	确定、易重复
Temperature	先 `z/T` 再 softmax	控制随机性
Top-k	只在概率最高 k 个中采样	k 固定，不适应分布形状
Top-p（nucleus）	取累计概率 ≥ p 的最小集合	集合大小随尖锐程度自适应

原文的实践结论：**Temperature + Top-p 的组合最常用**（如 temperature=0.7, top_p=0.9）。实现上通常先过滤 logits（被排除项设 -inf），再执行稳定 Softmax 和采样。

【补充】分布式推理的坑：若 vocab 被张量并行切分，**global top-k/top-p 需要跨设备聚合局部候选**，否则每个 rank 只在本地词表片段里采样，结果不正确。

第 12 章 Tokenization 与词嵌入

12.1 两道变换

```
文字 → ① Tokenization → token ids → ② Embedding 查表 → 向量矩阵
```

- **切分问题**：怎么把连续文字拆成可处理的"单元"；
- **编码问题**：怎么把每个单元表示成能捕捉语义关系的数字向量。

12.2 三种切分粒度

粒度	"Tokenization" 切成	词表大小	问题
字符级	T/o/k/e/n/i/z/a/t/i/o/n（12 个）	很小	序列过长，语义粒度太细
词级	Tokenization（1 个）	巨大	未登录词（OOV）无法处理
子词级	Token / ization（2 个）	中等	**主流选择**

12.3 BPE：数据驱动的合并

原文的关键点：**BPE（Byte Pair Encoding）从字符出发，把高频字符组合逐步合并成更大的 token**。直觉是"高频组合合并为单一 token，低频组合保留为子词拼接"——常见词（play、run）是完整 token，生僻词（tokenization）拆成 token + ization，**不会出现 OOV**。

原文的重要限定：**BPE 的切分是纯数据驱动的统计结果，不一定按语言学意义上的词根/后缀切分**，具体结果取决于训练词表。

模型	词表大小	分词策略
GPT-2 / GPT-3	50,257	BPE
LLaMA-2	32,000	SentencePiece BPE
LLaMA-3	128,256	Tiktoken BPE
GPT-4	~100,000	Tiktoken cl100k

原文的提醒：**词表越大覆盖越好，但 Embedding 矩阵也越大。LLaMA-3 把词表从 32K 扩到 128K，直接让 Embedding 层参数量增加了 4 倍。**

中文处理：大多数模型对中文做字符级 tokenization（每个汉字一个 token）或以 UTF-8 字节序列为依据；专门优化中文的模型（Qwen、ChatGLM）词表更大，包含更多中文词组作为单独 token。

12.4 独热编码为什么失败

问题	说明
维度爆炸	V 数万维稀疏向量，后续层需要极大权重矩阵
语义丢失	所有 token 两两正交，`猫·狗 = 猫·鱼 = 0`，模型看来"猫狗关系 = 猫鱼关系"

12.5 Embedding：一张可学习的查找表

```
W_emb ∈ R^{V × d_model}
Token ID = 1234 → W_emb[1234, :] → d_model 维向量
参数量 = V × d_model
```

实现上是查表（index lookup），等价于对独热向量做线性层，但效率高得多。

模型	V	d_model	压缩比
BERT-base	30,522	768	~40:1
LLaMA-2-7B	32,000	4,096	~8:1
GPT-3	50,257	12,288	~4:1

几何属性（原文强调这是训练涌现的，不是人工设计）：`king − man + woman ≈ queen`，以及 `walking − walk ≈ swimming − swim`——**语义关系被编码成几何关系。**

原文的一个重要澄清：在静态 Word Embedding 中，"银行"只有一个固定向量，落在金融与河岸两个语境之间；**真正消除歧义是 Self-Attention 的工作**——它根据上下文动态调整表示。**Embedding 提供起点，Self-Attention 完成动态消歧。**

12.6 完整输入流程与 Infra 视角

```
"猫在桌子上"
  → Tokenizer → [3421, 528, 11082, 319]        (4,)
  → Embedding → e_1..e_4, 堆成 X: (4, d_model)
  → + PE(0..3) → X: (4, d_model)               ← 位置信息（第 8 章）
  → 送入第 1 层 Decoder Block
```

三个 Infra 相关的判断：

判断	依据
Embedding 层通常不量化	它是查表操作，量化收益有限但精度损失较大
Weight Tying 能省一份 V×d 显存	GPT-2 采用；LLaMA-2 未采用（合计 2×131M ≈ 262M）
词表大小直接影响并行切分	LM Head 的 V 维常按 TP 切分，V 需能被 TP 整除

第 13 章 PyTorch：Tensor、autograd、训练循环与显存 / 性能分析

13.1 Tensor 的四个必会面向

面向	关键点	Infra 视角
创建	`randn/zeros/ones/arange/from_numpy`	`from_numpy` 共享内存，改一个另一个也变
shape 操作	`view/reshape/permute/squeeze/unsqueeze`	`view` 要求连续；`permute` 只改 stride
设备管理	`.to('cuda')` / `.cuda()` / `.cpu()`	走 PCIe，**频繁搬运是隐蔽瓶颈**
dtype	fp32 / fp16 / bf16	直接决定显存与算力（第 3 章）

```
t = torch.randn(2, 8, 12, 64)          # (batch, seq, heads, dim)
t2 = t.permute(0, 2, 1, 3)             # (batch, heads, seq, dim)
x = torch.randn(3, 4, device='cuda')   # 直接在 GPU 创建，避免多余搬运
x_bf16 = x.to(torch.bfloat16)
```

原文的提醒：**AI Infra 视角下，`freqently .cpu()` 和 `.cuda()` 往往是隐蔽的性能瓶颈**，第 13.5 节会用 profiler 抓这类问题。

13.2 autograd：三句话记住

1. `requires_grad=True` 让 PyTorch 追踪所有操作，前向时**动态建图**（Define-by-Run）；
2. `backward()` **只能对标量调用**；叶节点才保存梯度（`.grad`）；
3. **PyTorch 默认累加梯度，不自动清零**——所以每个 step 必须 `optimizer.zero_grad()`。

```
x = torch.tensor([1.0], requires_grad=True)
(x * 2).sum().backward()
print(x.grad)      # tensor([2.])
(x * 3).sum().backward()
print(x.grad)      # tensor([5.]) ← 2+3，梯度被累加！
x.grad.zero_()
```

这个设计是**有意为之**：梯度累积正是显存不足时模拟大 batch 的常用技巧（累积 K 步再 step 一次，等效 batch × K）。

13.3 nn.Module 与 state_dict

```
model = nn.Linear(10, 5)
for name, p in model.named_parameters():
    print(name, p.shape)                    # weight [5,10], bias [5]
print(sum(p.numel() for p in model.parameters()))   # 55
print(model.state_dict().keys())            # odict_keys(['weight','bias'])
```

概念	是什么	用途
`parameters()`	可训练参数迭代器	传给 optimizer
`named_parameters()`	带名字的参数	分层设学习率、调试
`state_dict()`	参数字典（不含结构）	保存/加载、跨框架转换
`module.children()`	子模块	逐层处理（如量化、切分）

关键区分：`state_dict` 只存参数，不存模型结构。 加载时必须先构造出同结构的模型，否则 `load_state_dict` 会报 key 不匹配。

13.4 标准训练循环（背下来）

```
for epoch in range(num_epochs):
    model.train()
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        loss = criterion(model(images), labels)   # forward
        loss.backward()                           # backward
        optimizer.step()                          # 更新
        optimizer.zero_grad()                     # 清零
    scheduler.step()                              # 每 epoch 调一次
```

DataLoader 的两个关键参数（原文点明）：

参数	作用	不设的后果
`num_workers`	多进程预取	GPU 等数据（data loading bottleneck）
`pin_memory=True`	锁页内存，走异步 DMA	CPU 拷贝开销进入关键路径

Checkpoint 要存**三样**（只存权重无法断点恢复）：

```
torch.save({
    'epoch': epoch,
    'model_state_dict': model.state_dict(),
    'optimizer_state_dict': optimizer.state_dict(),   # 少了它，Adam 动量丢失
    'loss': avg_loss,
}, 'checkpoint.pt')
```

13.5 显存与性能分析

```
print(torch.cuda.memory_allocated() / 1024**3)      # 当前已分配
print(torch.cuda.max_memory_allocated() / 1024**3)  # 峰值
print(torch.cuda.memory_summary(abbreviated=True))  # 分块视图

torch.cuda.reset_peak_memory_stats()                # 只重置计数器，不释放显存
```

原文给了两个极易踩的坑（第 5.3 节的注释），值得原样保留：

坑	现象	正解
`reset_peak_memory_stats()` 不释放显存	上一轮对象还活着，被算进本轮峰值，出现"负节省"	先 `del model, opt` 再 `empty_cache()`
混合精度只省激活	小 batch 时权重+优化器占主导，测出"节省 0%"	放大 batch / 序列长度让激活占主导

原文的参考实测：`FP32: 1881.6 MB | BF16: 1246.6 MB | 节省 33.7%`（大 batch 场景）。

profiler：

```
from torch.profiler import profile, record_function, ProfilerActivity

with profile(activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
             record_shapes=True, profile_memory=True) as prof:
    with record_function("forward"):
        loss = criterion(model(d), l)
    with record_function("backward"):
        loss.backward()

print(prof.key_averages().table(sort_by="cuda_time_total", row_limit=15))
prof.export_chrome_trace("trace.json")     # chrome://tracing 可视化
```

读输出的三条判据：

现象	结论
CPU total 远大于 CUDA total	GPU 在等 CPU（数据预处理、Python 开销）
大量小 CUDA kernel	launch overhead 累积，考虑 `torch.compile` 或算子融合
频繁 CPU–GPU 同步	`.item()` / `print(tensor)` 阻塞流水线

原文的实践建议：**不要每步都 `loss.item()`，每 100 步记录一次**。另外 backward 正常约为 forward 的 2 倍耗时，偏离这个比例就值得查。

第 14 章 GPU 硬件：SM、Tensor Core、存储层次与互联拓扑

14.1 CPU vs GPU：延迟导向 vs 吞吐导向

原文的"批改 10000 份试卷"类比：CPU 请一位经验丰富的老师（快、能处理复杂题型，但一次一份）；GPU 请 10000 个临时工（只会对答案，但同时开工）。

维度	CPU	GPU
设计目标	低延迟处理复杂任务	高吞吐处理大量简单任务
核心数量	几个到几十个大核	数千到数万个小核
单核能力	强（分支预测、乱序执行）	弱（简单 ALU，按序执行）
缓存占比	芯片面积大部分	芯片面积小部分
控制逻辑	复杂	简单（大量核心共享）
内存带宽	较低（DDR5 ~100 GB/s 量级）	极高（HBM3 ~3 TB/s 量级）
适合	串行逻辑、OS、网络服务	矩阵运算、数据并行、渲染

深度学习为什么天然适合 GPU？因为它拆到底层就是海量矩阵乘 + 逐元素运算，具有两个特征：**数据并行度极高**（每个元素独立）、**计算模式规则**（无复杂分支）。

原文的量级对比：**一块 H100 的 FP16 算力约 989 TFLOPS，而高端服务器 CPU 的 FP16 算力通常在个位数 TFLOPS 级别——差距在两个数量级以上。**

14.2 层级结构：GPC → TPC → SM → Core

```
GPU 芯片
├── GPC (Graphics Processing Cluster)     ← "事业部"
│   ├── TPC (Texture Processing Cluster)  ← "部门"
│   │   ├── SM (Streaming Multiprocessor) ← "小组", 基本调度单元
│   │   │   ├── CUDA Core × N   执行 FP32/INT32
│   │   │   ├── Tensor Core × M 执行矩阵乘累加
│   │   │   ├── SFU             sin/cos/exp
│   │   │   ├── Register File   寄存器堆
│   │   │   ├── Shared Memory   共享内存
│   │   │   └── L1 Cache
│   │   └── SM ...
│   └── TPC ...
├── GPC ...
├── L2 Cache（全局共享）
└── Memory Controller → HBM（显存）
```

以 H100 的 SM 为例：

组件	数量	功能
CUDA Core（FP32）	128	浮点与整数运算
Tensor Core（第四代）	4	矩阵乘累加（MMA）
Load/Store Unit	32	内存读写
SFU	16	超越函数
Warp Scheduler	4	每周期调度 4 个 Warp
Register File	256 KB	线程私有寄存器
Shared Memory / L1	228 KB（可配置）	SM 内共享高速存储

关键点：SM 是资源分配的最小粒度。 一个 Thread Block 被调度到一个 SM 上执行；SM 内的寄存器、共享内存由该 SM 上所有 Block 共同分配。这就是为什么"共享内存超限"会导致 occupancy 骤降。

14.3 Warp：GPU 执行的最小单位

GPU 不是一个线程一个线程执行，而是以 **Warp（32 个线程）** 为单位锁步执行同一条指令（SIMT，Single Instruction Multiple Threads）。

```
Warp（32 线程）
├── Thread 0:  add r1, r2, r3
├── Thread 1:  add r1, r2, r3   ← 同一条指令，不同数据
├── ...
└── Thread 31: add r1, r2, r3
```

Warp Divergence：Warp 内线程遇到分支走不同路径时，分支会被串行化、部分线程被掩码空转。**这是 GPU 编程中要极力避免的性能杀手。**

14.4 存储层次与 Memory Wall

以 H100 SXM 为例（带宽为整卡聚合值）：

层级	容量	带宽	延迟	作用域
Register	每线程 ~255 个	—	~0 cycle	线程私有
Shared Memory	每 SM ~228 KB	~31 TB/s【量级】	~30 cycle	SM 内所有线程
L1 Cache	与 SMEM 共享物理 SRAM	与 SMEM 合计 ~31 TB/s	~30 cycle	SM 内
L2 Cache	整卡 ~50 MB	~9.45 TB/s【量级】	~200 cycle	全局共享
HBM	80 GB	3.35 TB/s	~600 cycle	主存储

原文对带宽数字的口径说明非常严谨，值得照抄：

- Shared Memory 的 ~31 TB/s 是**理论估算**（每 SM 32 bank × 4 B/cycle = 128 B/cycle，132 SM × 128 B × 1.83 GHz ≈ 31 TB/s），前提是没有 Bank Conflict；
- L1 和 Shared Memory 是**同一块物理 SRAM**，带宽共享，不能当作 L1 单独带宽；
- L2 的 9.45 TB/s 取自 TileLang 论文附录，**不同测试方法差异较大，只作量级参考**；
- HBM3 的 3.35 TB/s 是 NVIDIA 官方理论峰值。

【补充】这张表最重要的信息不是具体数字，而是**层级之间差了 2～3 个数量级**（HBM 600 cycle vs SMEM 30 cycle）。所谓"高性能 kernel"，本质就是把数据尽可能往上面几层搬，并让每次搬运被尽可能多的计算复用。

14.4.1 Roofline 与算术强度

```
算术强度 = 浮点运算量（FLOPs）/ 内存访问量（Bytes）
算术强度 < 算力/带宽比  ⇒ Memory Bound
算术强度 > 算力/带宽比  ⇒ Compute Bound
```

H100 SXM 的 ops:byte 比：

```
989e12 / 3350e9 ≈ 295 FLOPs/Byte
```

原文的解读：**每从显存读 1 Byte 数据，需要执行至少 295 次 FP16 运算才能让算力跑满**，否则 GPU 会"闲着等数据"。

原文的结论也是整个推理优化的出发点：**大多数深度学习算子（尤其 Attention、LayerNorm、激活）的算术强度远低于 295**，所以 FlashAttention、Kernel Fusion 的核心思路都是减少显存访问、提升算术强度。

14.5 Tensor Core 与精度格式

精度	位宽	动态范围	首次 Tensor Core 支持	典型应用
FP32	32	高	—	基准、loss 累加
TF32	19	同 FP32	Ampere（A100）	训练（Ampere 上默认 FP32 路径）
FP16	16	较窄	Volta（V100）	混合精度（需 loss scaling）
BF16	16	同 FP32	Ampere（A100）	混合精度（推荐，无需 loss scaling）
FP8 (E4M3/E5M2)	8	可选	Hopper（H100）	训练 + 推理
INT8	8	整数	Turing（T4）	推理量化
FP4	4	极窄	Blackwell（B200）	推理量化

H100 SXM 各计算路径的稠密峰值（原文统一了口径）：

计算路径	理论峰值（约）
CUDA Core，FP32 FMA	67 TFLOPS
Tensor Core，TF32 输入	495 TFLOPS
Tensor Core，FP16/BF16 输入	989 TFLOPS
Tensor Core，FP8 输入	1,979 TFLOPS

原文的两条重要限定：① 这些是不同精度/执行路径的硬件峰值，**不能直接当作应用加速比**；② NVIDIA 产品页带星号的 Tensor Core 数值包含稀疏加速，上表已除以 2 取稠密口径。**读厂商规格时必须逐项确认是否标注 "with sparsity"。**

对齐要求：要触发 Tensor Core 加速，矩阵维度必须满足对齐（通常 8 或 16 的倍数）。**如果 hidden_size 不是 8 的倍数，Tensor Core 可能无法被充分利用**——这也是第 7 章 `d_ff=11008`（256 的倍数）的工程理由。

14.6 主流 AI GPU 规格对比

型号	架构	SM 数	显存	带宽	FP16 算力	FP8 算力	TDP
V100	Volta	80	32 GB HBM2	900 GB/s	125 TFLOPS	—	300W
A100 SXM	Ampere	108	80 GB HBM2e	2,039 GB/s	312 TFLOPS	—	400W
H100 SXM	Hopper	132	80 GB HBM3	3,350 GB/s	989 TFLOPS	1,979 TFLOPS	700W
H200	Hopper	132	141 GB HBM3e	4,800 GB/s	989 TFLOPS	1,979 TFLOPS	700W

型号	架构	SM 数	显存	带宽	FP16 算力	FP8 算力	TDP
B200	Blackwell	160	192 GB HBM3e	8,000 GB/s	—【见下注】	—【见下注】	1000W
L40S	Ada	142	48 GB GDDR6	864 GB/s	366 TFLOPS	—	350W

> **B200 数字口径提醒**：AIInfraGuide 的两篇文章对 B200 的表述不一致——`gpu-basics` 的对比表把 FP8 写为 4,500 TFLOPS；`nvidia-gpu-evolution` 明确写 **FP8 Dense 2,250 / 稀疏 4,500**，并给出 FP16 约 1,125 TFLOPS（是 FP8 的一半）。两个口径差别正是"稠密 vs 稀疏"。**本笔记采用 evolution 文章的稠密口径（2,250），并把差异标注在此，不做取舍性断言。** 引用 B200 峰值的任何场合，都必须先说明是否含稀疏。

封装差异：同一架构可能有 SXM 和 PCIe 两种封装。SXM 版本通常功耗更高、频率更高、且支持 NVLink 互联，性能显著强于 PCIe 版本。**选型时务必确认封装类型**（H100 PCIe 版虽然也有 NVLink 4.0，但仅支持 2 卡桥接互联）。

14.7 Volta → Blackwell 五代演进

架构	年份	代表	核心突破
Volta	2017	V100	首次引入 Tensor Core（4×4×4 混合精度 MMA）
Turing	2018	T4	INT8/INT4 推理加速、RT Core
Ampere	2020	A100	第三代 Tensor Core、MIG、TF32、BF16、2:4 稀疏
Hopper	2022	H100	FP8、Transformer Engine、NVLink 4.0、TMA、Thread Block Cluster
Blackwell	2024	B200/GB200	第五代 Tensor Core、NVLink 5.0、双芯封装、FP4

三条演进主线（原文总结）：

1. **精度格式不断下探**：FP32 → FP16 → BF16/TF32 → FP8 → FP4，每压一级理论算力翻倍，同时需要配套数值稳定性方案（AMP、Transformer Engine）；
2. **互联带宽持续翻倍**：NVLink 300 → 600 → 900 → 1,800 GB/s，连接范围从节点内 8 卡扩展到 72 卡；
3. **软硬件协同深化**：从 Tensor Core 硬件加速，到 Transformer Engine 自动精度调度，再到 MIG 虚拟化隔离。

几个值得单独记的点：

技术	机制	工程意义
TF32	FP32 的 8 位指数 + FP16 的 10 位尾数（19 bit）	Ampere 上默认开启，FP32 代码自动提速

技术	机制	工程意义
MIG	把一块 A100 物理隔离为最多 7 个实例	多租户推理、多人共用开发卡；不适合大模型训练
Transformer Engine	每层计算前分析张量分布，动态决定 FP8 还是 BF16	"自动挡"：数值稳定的层挂高速档
TMA	硬件异步内存拷贝单元	把数据搬运与计算解耦，减少 SM 空闲
双芯封装（B200）	两块 die 通过 10 TB/s 片间互联封装	对外是一块 GPU，算力翻倍

14.8 多卡互联：NVLink / NVSwitch / PCIe / IB

互联技术	双向带宽	场景	说明
PCIe 4.0 x16	64 GB/s	跨卡通用	单向 32 GB/s
PCIe 5.0 x16	128 GB/s	跨卡通用	单向 64 GB/s
NVLink 3.0（A100）	600 GB/s	机内	12 链路 × 50 GB/s
NVLink 4.0（H100）	900 GB/s	机内	18 链路 × 50 GB/s，单向 450 GB/s
NVLink 5.0（B200）	1,800 GB/s	机内	18 链路 × 100 GB/s
NVSwitch	连接所有 NVLink	机内全互联	任意两卡满带宽，消除拓扑瓶颈
InfiniBand NDR	400 Gb/s ≈ 50 GB/s	跨机	大规模集群标配
InfiniBand XDR	800 Gb/s ≈ 100 GB/s	跨机	更新一代

原文的对比结论：**NVLink 4.0 是 PCIe 5.0 的约 7 倍（900 vs 128 双向），NVLink 5.0 约为 14 倍。** 这个差距直接决定了一条工程准则：**需要高频通信的并行策略（如 TP）必须限制在 NVLink 互联范围内。**

14.8.1 典型拓扑：DGX H100（8 卡）

```
8 × H100 ──NVSwitch── 任意两卡 900 GB/s 全互联
   │
8 × ConnectX-7 IB 网卡 ──→ 集群
```

原文的关键判断：**机内通信 >> 机间通信**。所以在设计并行策略时，**应该让通信量大的维度（TP）放在 NVLink 上，通信量相对小的维度（DP、PP）放在 IB 上。**

14.8.2 读 `nvidia-smi topo -m`

```
        GPU0    GPU1    GPU2    GPU3    ...
GPU0     X      NV18    NV18    NV18    ...
GPU1    NV18     X      NV18    NV18    ...
GPU2    NV18    NV18     X      NV18    ...
```

标记	含义
`NV18`	通过 18 条 NVLink 连接（NVLink 4.0 满配）
`NV12`	通过 12 条 NVLink 连接（A100 满配）
`PIX`	同一 PCIe 交换机下（跨 1 个 PCIe switch）
`PHB`	通过 PCIe Host Bridge 连接
`NODE`	同一 NUMA 节点内，通过 PCIe 连接
`SYS`	跨 NUMA 节点，需经过系统总线（最慢）

判读方法：对角线是 `X`（自己）。看到 `NV*` 说明走 NVLink；看到 `SYS` 说明这条路径要跨 NUMA，**这是 TP 组最差的落点**。

```
nvidia-smi topo -m          # 拓扑矩阵
nvidia-smi nvlink -s        # NVLink 状态与速率
nvidia-smi mig -lgip        # 查看 A100 支持的 MIG 配置
```

14.9 显存管理策略速查

策略	原理	省什么	代价
混合精度	前向反向低精度，更新高精度	~50% 参数显存	FP16 需 loss scaling
梯度累积	多 micro-batch 累积再 step	激活峰值	训练步数增加
梯度检查点	只存部分激活，反向重算	激活降到 O(√N) 量级	约 +33% 计算
ZeRO	优化器状态/梯度/参数分片	线性降低每卡	通信量增加
Offloading	卸到 CPU 内存 / NVMe	突破单卡上限	PCIe 带宽成为瓶颈

原文的实践组合：**ZeRO Stage 2 + 混合精度 + 梯度检查点** 是 7B～70B 训练的常见配置。

第 15 章 集合通信：原语语义、通信量公式、Ring/Tree 与 NCCL

15.1 为什么通信决定并行策略的形状

原文的定位：集合通信是分布式训练的"血管系统"。单卡再强也有天花板，GPU 之间"怎么说话、说话多快"决定整体效率。

先记住一条带宽阶梯（第 14 章的表，这里只保留用得到的）：

```
NVLink 4.0 (H100)   900 GB/s 双向  ≈ 450 GB/s 单向
PCIe 5.0 x16        128 GB/s 双向  ≈  64 GB/s 单向
IB NDR 400Gb/s                     ≈  50 GB/s 单向
```

机内比机间快约 9 倍——这一个比例就能解释大部分并行策略的设计选择。

15.2 单机：NVLink 与 NVSwitch

互联	单链路（双向）	每 GPU 链路数	每 GPU 总带宽（双向）
NVLink 3.0（A100）	50 GB/s	12	600 GB/s
NVLink 4.0（H100）	50 GB/s	18	900 GB/s
NVLink 5.0（B200）	100 GB/s	18	1,800 GB/s

NVSwitch 的价值是**消除拓扑瓶颈**：8 卡若两两直连，布线复杂度爆炸；通过交换芯片，任意两卡都能跑满带宽。**这对 TP 至关重要——TP 每层都要 AllReduce，如果某些卡对带宽偏低，整体就被最慢链路拖累。**

15.3 多机：InfiniBand 与 RoCE

普通以太网（TCP/IP）的三个硬伤（原文）：延迟高（内核协议栈 50–100 μs）、CPU 开销大、有效带宽受限。

RDMA（Remote Direct Memory Access）解决它：零拷贝、内核旁路、微秒级延迟（1–2 μs，比 TCP/IP 快 50–100 倍）。

标准	单链路	4x 常用配置	等效字节带宽
HDR	50 Gb/s	200 Gb/s	25 GB/s
NDR	100 Gb/s	400 Gb/s	50 GB/s
XDR	200 Gb/s	800 Gb/s	100 GB/s

维度	InfiniBand	RoCE v2
延迟	~1 μs	~2–5 μs

维度	InfiniBand	RoCE v2
带宽	400–800 Gb/s	100–400 Gb/s
拥塞控制	硬件原生	依赖 ECN/PFC 软件配置
成本	高（专用设备）	中（复用以太网）
大规模稳定性	成熟	需仔细调优
适用	大型 AI Lab、万卡集群	云厂商、中小规模

选型建议：追求极致性能与大规模稳定性 → IB；预算受限或复用现有网络 → RoCE v2。**两者对上层训练代码完全透明，NCCL 会自动选择传输方式。**

15.4 五大集合通信原语与通信量公式

设总数据量为 M，参与节点数为 N。

原语	语义	每 rank 发送量	每 rank 接收量	典型用途
Broadcast	一对多	M（root）/ 0	0（root）/ M	参数初始化广播
Reduce	多对一聚合	0（root）/ M	M（root）/ 0	Loss 汇总
AllReduce	聚合后所有人都有	≈2M	≈2M	DDP 梯度同步
AllGather	每人一片，最后人人全有	(N−1)M/N	(N−1)M/N	ZeRO-3 收集参数
ReduceScatter	归约后各取一片	(N−1)M/N	(N−1)M/N	ZeRO 梯度分片
All-to-All	每人对每人的不同分片	视切分而定	视切分而定	MoE 的 EP、序列并行

15.4.1 数据流示意（4 卡，数据 [a,b,c,d]）

```
AllReduce(sum):  每卡 [1,2,3] 等 → 每卡都拿到 [10,20,30]
AllGather:       GPU0:[A] GPU1:[B] GPU2:[C] GPU3:[D] → 每卡 [A,B,C,D]
ReduceScatter:   每卡 [1,2,3,4] → GPU0:[4] GPU1:[8] GPU2:[12] GPU3:[16]
```

核心恒等式：`AllReduce = ReduceScatter + AllGather`。 这正是 Ring AllReduce 的算法基础。

15.5 Ring AllReduce：带宽最优

15.5.1 两个阶段

```
阶段一 ReduceScatter (N-1 轮):
  每 rank 把数据均分为 N 块；每轮向下一个 rank 传一块，
  同时接收上一个 rank 传来的块并与本地对应块累加
  → 经过 N-1 轮，每个 rank 持有完全归约后的 1/N 数据块

阶段二 AllGather (N-1 轮):
  每个 rank 把自己已归约好的那块沿环传递
  → 最终每个 rank 都拥有完整结果
```

15.5.2 通信量公式（必背）

每个 rank 在 ReduceScatter 阶段发送 (N−1) 次、每次 M/N 字节；AllGather 同理：

```
通信量 = 2 × (N-1) × M/N = 2(N-1)/N × M
```

当 N 较大时 `(N-1)/N → 1`，通信量趋近 **2M，与节点数基本无关**。这就是 Ring AllReduce 被称为**带宽最优（bandwidth-optimal）**的原因。

但它不是延迟最优：`2(N-1)` 轮传递意味着延迟随节点数线性增长，在节点多而数据量小的场景会成为瓶颈。

15.6 Tree AllReduce：延迟最优

Ring 的延迟是 O(N)。数据量很小时（如同步一个标量 loss），延迟而非带宽才是瓶颈。Tree AllReduce 用树形拓扑把延迟降到 O(log N)。

对比项	Ring AllReduce	Tree AllReduce
端到端延迟	O(N)	O(log N)
带宽利用率	接近 100%（大数据量）	较低（根节点是瓶颈）
最适场景	梯度同步（大张量）	标量指标同步（loss、step）

原文的关键提示：**NCCL 内部会根据数据大小自动在 Ring 和 Tree 之间切换**（也可用 `NCCL_ALGO=Ring|Tree` 强制），该决策对上层框架完全透明。

15.7 通信与计算 Overlap

理论最优是：计算与通信完全并行，通信延迟被"藏进"计算时间。核心思想：**在反向传播计算后续层梯度的同时，并行传输已就绪的前面层梯度。**

PyTorch DDP 通过 **bucket** 机制自动实现：参数被分组为若干 bucket，一个 bucket 内所有梯度算完就立即异步触发 AllReduce。

```
model = DDP(model, device_ids=[local_rank], bucket_cap_mb=25)
```

`bucket_cap_mb` 的权衡（原文）：**太大** → 等待时间长（需积累更多梯度才触发）；**太小** → 通信次数多（每次有固定启动开销）。**通常 25MB 是好的起点。**

15.8 NCCL：自动挡 + 可观测性

NCCL（NVIDIA Collective Communications Library，读作 "Nickel"）是 PyTorch、DeepSpeed、Megatron-LM 底层多卡通信的驱动。核心能力：自动拓扑感知、全套集合通信、单机多机透明、与 CUDA Stream 配合做重叠。

传输后端选择（自动）：

传输方式	适用场景	说明
NVLink / NVSwitch	单机卡间	最高带宽，优先
PCIe P2P	单机卡间（无 NVLink）	带宽较低
InfiniBand Verbs	多机	RDMA，低延迟高带宽
RoCE	多机	以太网上的 RDMA
Socket (TCP)	回退	性能最差，仅兼容

```
tensor = torch.ones(1024, 1024, device=f"cuda:{dist.get_rank()}")
dist.all_reduce(tensor, op=dist.ReduceOp.SUM)

local_chunk = torch.randn(256, device=f"cuda:{dist.get_rank()}")
gathered = [torch.zeros(256, device=f"cuda:{dist.get_rank()}") for _ in range(world_size)]
dist.all_gather(gathered, local_chunk)

output = torch.zeros(1024 // world_size, 1024, device=f"cuda:{dist.get_rank()}")
dist.reduce_scatter(output, list(tensor.chunk(world_size)), op=dist.ReduceOp.SUM)
```

15.8.1 关键环境变量

变量	说明	常用值
`NCCL_DEBUG`	日志级别	`VERSION` / `WARN` / `INFO` / `TRACE`
`NCCL_DEBUG_SUBSYS`	过滤子系统	`ALL` / `NET` / `INIT`
`NCCL_TOPO_DUMP_FILE`	导出拓扑 XML	`/tmp/nccl_topo.xml`
`NCCL_ALGO`	强制算法	`Ring` / `Tree`
`NCCL_PROTO`	强制协议	`LL` / `LL128` / `Simple`

变量	说明	常用值
`NCCL_SOCKET_IFNAME`	指定 TCP 网卡	`eth0` / `^lo`
`NCCL_IB_HCA`	指定 IB 设备	`mlx5_0`
`NCCL_NET_GDR_LEVEL`	GPUDirect RDMA 级别	0（禁用）～5
`NCCL_P2P_DISABLE`	禁用 P2P	`1`
`NCCL_BUFFSIZE`	通信缓冲区	默认 4MB，可设 16777216

```
NCCL_DEBUG=INFO python train.py                          # 看拓扑与算法选择
NCCL_IB_HCA=mlx5_0 NCCL_NET_GDR_LEVEL=2 python train.py  # IB + GPUDirect
NCCL_ALGO=Ring NCCL_DEBUG=INFO python train.py           # 强制 Ring 对比
```

15.8.2 用 nccl-tests 测真实带宽

```
git clone https://github.com/NVIDIA/nccl-tests.git && cd nccl-tests
make MPI=1 MPI_HOME=/usr/local/mpi CUDA_HOME=/usr/local/cuda NCCL_HOME=/usr/local/nccl
./build/all_reduce_perf -b 8 -e 256M -f 2 -g 8        # 单机 8 卡
mpirun -np 16 --hostfile hosts -x NCCL_DEBUG=INFO \
       ./build/all_reduce_perf -b 8 -e 256M -f 2 -g 8   # 多机
```

输出中的两个关键列：

```
   size(B)  count  type  redop   time(us)  algbw(GB/s)  busbw(GB/s)
 268435456 ...    float   sum      5210.0          51.5         90.1
```

指标	定义	用途
`algbw`	数据量 / 时间	应用层看到的吞吐
`busbw`	`algbw × 2(N-1)/N`	**修正算法传输倍数，反映硬件链路真实利用率**

对照硬件理论带宽时要看 busbw，不是 algbw。 原文给的经验参考值（8×H100 SXM 单机 AllReduce）：**busbw 应与 NVLink 4.0 的单向 450 GB/s 对照（而非双向 900 GB/s），实测通常在 ~360–480 GB/s 量级**（开启 NVLS 时可略高于 450 GB/s）。

15.9 从通信视角解释并行策略

15.9.1 TP 为什么只能待在机内

一个 Transformer 层：Attention 和 MLP 各前向、反向各 1 次 AllReduce，**合计 4 次/层**。80 层就是 **320 次**。

假设每次 AllReduce 搬运 1GB（统一按单向带宽算）：

路径	单次耗时	320 次总计
NVLink 4.0（单向 450 GB/s）	约 2.2 ms	约 0.71 s
IB NDR（单向 50 GB/s）	约 20 ms	约 6.4 s

差距约 9 倍，训练速度断崖式下降。这就是"TP 不跨机"的物理原因。

15.9.2 DP 的通信开销

AllReduce 数据量 = 模型梯度大小：

模型	FP16 梯度	8 卡 Ring AllReduce 每卡	NVLink 4.0 耗时
7B	14 GB	~28 GB	~31 ms
70B	140 GB	~280 GB	~311 ms

若跨机走 IB（50 GB/s），70B 的 AllReduce 约 5.6 秒——**这就是 ZeRO 要把梯度和参数切碎分片的动机：用更多次小通信替代一次大通信。**

15.9.3 PP 的气泡

```
气泡占比 = (P-1) / (P-1+M)      P = 流水线级数，M = micro-batch 数
```

增大 M 能缩小气泡，但增加显存占用。**PP 的通信量远小于 TP 和 DP，IB 完全够用，代价是气泡。**

15.9.4 并行策略 ↔ 通信原语映射

并行策略	主要原语	频率	数据量
DDP	AllReduce	每层反向后	梯度大小
ZeRO-1/2	ReduceScatter + AllGather	反向后 + 更新后	梯度大小
ZeRO-3	AllGather + ReduceScatter	每层前向 + 反向	参数大小
TP	AllReduce / AllGather	每层前向 + 反向	激活大小
PP	点对点 Send/Recv	每 micro-batch	激活大小
MoE / EP	All-to-All	每层	token 路由量

15.10 通信瓶颈排查清单

现象	可能原因	排查方法
step 时间远超预期	通信带宽不足	`nccl-tests` 测实际带宽
GPU 利用率长期偏低	数据加载或 CPU 预处理	Nsight Systems 看 CPU–GPU 交互
不同卡速度不一致	NVLink 拓扑不对称	`nvidia-smi topo -m`
多机扩展效率差	IB 配置问题	`NCCL_DEBUG=INFO` 看传输路径
带宽远低于理论值	算法选择不优或缓冲区太小	`NCCL_ALGO` 强制切换对比
显存 OOM	KV Cache / 激活 / 优化器超限	逐项计算（第 10 章）

第 16 章 六个模块的检验标准（源自学习路线 0.3 节，逐条转为可自测项）

AllnfraGuide 学习路线第 0.3 节把第零层的门槛定成 7 条。本章把它们**逐条拆成可自测的动作**，并按 6 个模块归位。**判定原则：能不看资料、在白板/终端上当场完成，才算过。**

16.1 七条原始标准与模块归属

#	路线原文标准	归属模块	通过判据
1	Transformer 白板默写	Transformer	画出 Block 全结构 + 标维度 + 说清 Q/K/V 来源
2	维度推导（7B 参数量，误差 <20%）	Transformer	手算落到 6.6B～8.1B 区间内
3	PyTorch 训练脚本	PyTorch	不借 Trainer，GPU 上跑通含调度与 checkpoint
4	C++ 基础读写（CUDA host 端）	编程语言	读懂 malloc/memcpy/launch/free 四段
5	Linux 日常	编程语言	SSH + tmux + conda + nvidia-smi + git + bash
6	拓扑感知（`nvidia-smi topo -m`）	GPU 硬件	判出哪些卡走 NVLink、哪些走 PCIe
7	集合通信直觉（三种原语数据流 + 公式）	集合通信	画图 + 写公式 + 解释 NVLink/IB 差距的含义

16.2 模块一：编程语言基础

- [ ] 能在 10 分钟内解释清楚：`a=b` 的引用语义、默认可变参数为什么错、GIL 限制了什么/没限制什么
- [ ] 给定一个任务，能按"任务类型表"选出线程/进程/asyncio，并说出理由
- [ ] 能写出带 warmup + repeat + median 的 benchmark，并说明 GPU 为什么要 synchronize

- [ ] 能默写 `.cpp → .ii → .s → .o → 可执行` 四阶段命令，并区分编译/链接/装载三类错误
- [ ] 能解释 RAII 与 `unique_ptr` / `shared_ptr` 的取舍
- [ ] **（标准 4）** 能逐行讲清一段 CUDA host 代码：`cudaMalloc → cudaMemcpy(H2D) → kernel<<<grid,block>>> → cudaMemcpy(D2H) → cudaFree`，并说出 H2D/D2H 走哪条总线
- [ ] **（标准 5）** 能在服务器上独立完成：SSH 登录、`tmux` 建/脱离/恢复会话、`conda create/activate`、`nvidia-smi` 看进程、`git` 提交、写一个带 `set -euo pipefail` 的批量提交脚本
- [ ] 能解释 `nvidia-smi` 的 "CUDA Version" 为什么不等于 `nvcc --version`

16.3 模块二：数学基础

- [ ] 看到 `(B,S,H) @ (H,V)` 立刻写出 `(B,S,V)`；看到 `(B,N_h,S,D_h) @ (B,N_h,D_h,S)` 写出 `(B,N_h,S,S)`
- [ ] 能写出稳定 Softmax 与 LogSumExp，并验证 `softmax(z+c)=softmax(z)`
- [ ] 能写出 `H(q,p)=H(q)+D_KL(q‖p)`，并说明为什么最小化交叉熵等价于最小化 KL
- [ ] 能徒手推出 `∂L/∂z = p − y`
- [ ] 能写出线性层三个梯度的 shape 并自检
- [ ] 能区分精度与动态范围；能解释 FP16 的 65504 与 BF16 的"同 FP32 指数"
- [ ] 能解释并行归约为什么可能不逐位一致，以及 Kahan/Welford 解决什么
- [ ] 能用 `2·B·N_h·S²` 算出显式注意力矩阵的字节数

16.4 模块三：Transformer（权重最高的模块）

- [ ] **（标准 1）** 白板默写 Decoder Block：`x → RMSNorm → MaskedMHA → Add → RMSNorm → SwiGLU FFN → Add`，两条残差路径都从 Add 直拉
- [ ] 能标出每一步 shape：`(B,S,d) → (B,N_h,S,D_h) → (B,N_h,S,S) → (B,N_h,S,D_h) → (B,S,d)`
- [ ] 能说清 Q/K/V 是怎么从输入通过三个线性投影得到的，以及为什么要三个而不是一个
- [ ] 能画出 5×5 因果掩码（两种画法），并解释 `exp(−inf)=0`
- [ ] **（标准 2）** 手算 7B：`d=4096, h=32, L=32, V=32000, d_ff=11008`，得到 ≈6.74B（区间 5.4B～8.1B）
- [ ] 能说出 FFN 占单 Block 参数约 2/3，并给出 135M vs 67M
- [ ] 能写出 `d_ff = 8/3·d` 的推导，并解释为什么取 11008
- [ ] 能解释 MHA → GQA 省的是 KV Cache 而不是训练参数，并算出 16 KB → 4 KB
- [ ] 能解释 RoPE 为什么只作用于 Q/K
- [ ] 能解释 Pre-Norm 为什么比 Post-Norm 稳（用 `I + ∂F/∂x` 论证）
- [ ] 能默写 KV Cache 公式并算出 7B 在 seq=4096/batch=16 下约 32 GB

- [ ] 能用 `6Ns` 算训练 FLOPs、`2Ns` 算推理 FLOPs，并解释 0.043 ms 与 10–20 ms 的差距

16.5 模块四：PyTorch

- [ ] **（标准 3）** 不借 Trainer 写出完整训练循环：DataLoader（含 `num_workers` / `pin_memory`）→ forward → loss → backward → step → zero_grad → scheduler → checkpoint，并在 GPU 上跑通
- [ ] 能解释梯度为什么默认累加，以及梯度累积如何模拟大 batch
- [ ] 能说出 checkpoint 必须存 model/optimizer/epoch 三样，缺 optimizer state 的后果
- [ ] 能用 `memory_allocated` / `max_memory_allocated` / `memory_summary` 定位显存
- [ ] 能用 `torch.profiler` 抓一次 step，判断是 CPU 瓶颈、launch overhead 还是隐式同步
- [ ] 能用 `torch.autocast` 做 bf16 训练，并解释 GradScaler 只在 fp16 下需要
- [ ] 能复述"测混合精度显存时先 del + empty_cache"这个坑

16.6 模块五：GPU 硬件

- [ ] **（标准 6）** 在 8 卡机上跑 `nvidia-smi topo -m`，指出哪些卡对是 `NV18` / `NV12`、哪些是 `PHB` / `SYS`，并判断 TP 组该怎么放
- [ ] 能说出 A100/H100/H200 的 SM 数（108/132/132）、显存（80/80/141 GB）、带宽（2,039/3,350/4,800 GB/s）、FP16 稠密算力（312/989/989 TFLOPS）
- [ ] 能画出 GPC → TPC → SM → CUDA Core/Tensor Core
- [ ] 能解释 Warp 与 Warp Divergence
- [ ] 能默写存储层次五级的容量/带宽/延迟量级，并说明为什么 HBM 是瓶颈
- [ ] 能用 Roofline 判断算子类型，并算出 H100 的 ~295 FLOP/Byte
- [ ] 能解释 TF32/BF16/FP8/FP4 的位宽与首代支持架构
- [ ] 能解释"带星号的 TFLOPS 含稀疏"这个规格陷阱
- [ ] 能说出 5 种显存优化策略及各自代价

16.7 模块六：集合通信

- [ ] **（标准 7）** 画出 AllReduce / AllGather / ReduceScatter 三张数据流图，并写出各自通信量公式
- [ ] 能默写 Ring AllReduce 的 `2(N-1)/N × M`，并解释为什么与 N 基本无关
- [ ] 能对比 Ring（O(N) 延迟、带宽最优）与 Tree（O(log N) 延迟、根节点瓶颈）
- [ ] 能解释 `AllReduce = ReduceScatter + AllGather`
- [ ] 能说出 NVLink 4.0 / PCIe 5.0 / IB NDR 的单向带宽，并解释"TP 不跨机"
- [ ] 能用 `nccl-tests` 跑出 busbw，并说明该跟 450 GB/s 还是 900 GB/s 比

- [ ] 能解释 DDP bucket 的 overlap 机制与 `bucket_cap_mb` 权衡

第 17 章 与已有笔记的衔接：把前置知识接到四门课上

本工作区已有一批课程笔记。第零层的价值不在于"学完"，而在于**知道往下该接哪条线**。下面这张表给出精确的接口。

已有笔记	接本文哪一章	接口内容	建议用法
CSAPP 学习笔记	第 14 章存储层次	Cache 层级、局部性、内存层次金字塔	把 HBM→L2→SMEM→Register 当作 CSAPP 存储层次的"GPU 版"重读一遍
CS61C 学习笔记	第 14 章 GPU 架构	SIMD/SIMT、流水线、Roofline、Amdahl 定律	用 CS61C 的 Roofline 直接套 H100 的 295 FLOP/Byte
CMU 15-418 学习笔记	第 15 章集合通信	并行模式、归约、扫描、work stealing、通信成本模型	把 AllReduce 当作"并行归约"的分布式版本
CS149 项目速查	第 5、14 章	GPU 编程模型、tiling、occupancy	用 CS149 的 tiling 练习理解 FlashAttention 的分块
ECE408 学习笔记	第 14 章 + CUDA 层	CUDA 线程层级、共享内存、合并访存	第零层过关后，直接进 ECE408 写 kernel
CS336 学习笔记	第 4～12 章	语言模型从零实现、tokenizer、训练	把第 10 章的参数量手算与 CS336 的实现对照
CS144 学习笔记	第 15 章多机通信	拥塞控制、RTT、带宽时延积	理解 IB 的硬件拥塞控制 vs TCP 的端到端控制
MIT 6.172 性能工程	第 2、14 章	性能测量方法论、优化优先级、Cache 优化	用 6.172 的"先算法后常数"校准第 2.6 节的优先级
DeepSeek V4.1-Flash 解析	第 6、7、11 章	MLA、MoE、KV Cache 压缩	把架构变种放回真实模型里看

三条衔接建议：

1. **存储层次只学一遍，但要用两套语言描述**：CSAPP 的语言是 Cache line / 组相联 / 命中率；GPU 的语言是 bank / coalescing / occupancy。第 14 章的表就是翻译字典。
2. **并行模式要"上移一层"**：15-418 讲的是线程级并行，第 15 章讲的是**节点级并行**，但成本模型同构——都是"通信次数 × 单次延迟 + 数据量 / 带宽"。
3. **CS336 是唯一的"端到端"校验**：当你能把第 10 章的维度表和 CS336 的实现逐行对上，第零层就真的通了。

第 18 章 复习清单与自测题（附答案）

18.1 一页纸复习清单

主题	必须能"闭卷"输出的内容
全景	三层+第零层；不可能三角；5 个"牺牲什么换什么"
语言	GIL 三个例外；四阶段编译；`undefined reference` vs `cannot open shared object`
数学	`(B,S,H)@(H,V)`；稳定 Softmax；`p-y`；16B/参数
Attention	五步公式；$8Nd^2+4N^2d$；因果掩码；FlashAttention 一句话
变种	MHA/GQA/MQA 的 KV Cache 16/4/0.5 KB；MLA 压维度
FFN	2/3 参数；$8/3 \cdot d$；列切+行切+一次 AllReduce
位置	排列等变；Sinusoidal 公式；RoPE 只转 Q/K
归一化	Pre-Norm 梯度式；RMSNorm 少一次归约
Block	22 步维度表；6.74B 手算
生成	Prefill/Decode 瓶颈；KV Cache 公式；TTFT/TPOT
PyTorch	训练循环五步；checkpoint 三样；profiler 三判据
GPU	A100/H100/H200 三行规格；五级存储；295 FLOP/Byte
通信	五个原语；$2(N-1)/N \cdot M$；Ring vs Tree；algbw vs busbw

18.2 自测题（12 题，答案在后）

1. 一个 `(B,S,H)=(8,2048,4096)` 的张量过 `W:(4096,16384)` 的线性层，前向 FLOPs 是多少？
2. 为什么 `x.grad` 在两次 `backward()` 后会累加？这与梯度累积技巧是什么关系？
3. 写出 Ring AllReduce 每 rank 的通信量，并说明 N=64 时它约等于多少 M。
4. 一个 7B 模型用 AdamW + BF16 训练，静态显存（不含激活）是多少？为什么不是 14 GB？
5. `seq=32768, N_h=32, B=1, FP16` 时显式物化注意力分数矩阵需要多少显存？
6. GQA 把 KV 头从 32 降到 8，KV Cache 变成原来的几分之几？训练参数量会变吗？
7. 为什么 `nvidia-smi` 显示 CUDA 12.4，但 `nvcc --version` 可能是 11.8？
8. H100 的 ops:byte 比是多少？这个数字对写 kernel 有什么指导意义？
9. Pre-Norm 的梯度连乘里，哪一项保证了梯度不会消失？

10. `nccl-tests` 输出 `algbw=51.5, busbw=90.1`（8 卡），该跟 450 GB/s 还是 900 GB/s 比？结论是什么？
11. 为什么 TP 一般不出机？用两个数字说明。
12. FP16 的最大有限值是多少？这解释了为什么需要 Loss Scaling 吗？

18.3 答案

1. `2 × B × S × H × 4H = 8·B·S·H² = 8×8×2048×4096² ≈ 2.20×10¹²` FLOPs（按第 10.5.1 的 `2MKN` 公式复算）。

2. 因为一个张量可能通过多条路径影响 loss，反向时各路径的梯度要**相加**（第 3 章计算图例子 `∂L/∂x = 2b + 2by`）。PyTorch 把这个语义统一成"`.grad` 累加"，于是梯度累积技巧只需"多次 backward 后 step 一次"，无需任何额外机制；代价是每步必须 `zero_grad()`。

3. `2(N-1)/N × M`。N=64 时 `2×63/64 = 1.969M ≈ 2M`，**与节点数几乎无关**——这就是带宽最优的含义（第 15.5.2）。

4. `2N + 2N + 12N = 16N`，7B（6.74B）→ 约 **107.2 GB**。不是 14 GB，因为除 BF16 权重（13.4 GB）外还有 BF16 梯度（13.4 GB）、FP32 master weights（26.8 GB）、Adam 一阶/二阶动量（各 26.8 GB）（第 10.6.2）。若把梯度也按 FP32 存，则是 18 B/参数 ≈ 126 GB。

5. `2 × B × N_h × S² = 2×1×32×32768² = 64 GiB`（第 5.8 与第 11.4.2）。这还没算概率矩阵与反向中间量。

6. KV Cache 变为 **1/4**（16 KB → 4 KB 单 token，第 6.2）。**训练参数量会变**：K/V 投影从 `d×d` 变成 `d×(h_kv·d_k)`，W_K/W_V 各从 16M 降到 4M。注意"省 KV Cache"是主要动机，"省参数"是副产品。

7. `nvidia-smi` 的 "CUDA Version" 是**驱动支持的最高 CUDA 兼容级别**，不是已安装的 toolkit 版本；`nvcc --version` 才是编译器版本。二者可以不一致，且框架可能自带一套 CUDA runtime（第 2.10.6）。

8. `989e12 / 3350e9 ≈ 295 FLOPs/Byte`。含义：每从 HBM 读 1 Byte 至少要算 295 次 FP16 运算才不浪费算力；**算术强度低于 295 的算子就是 Memory Bound**，优化方向应是减少访存/提升复用，而不是堆算力（第 14.4.1）。

9. 连乘展开后**恒为单位阵 I 的那一项**（所有因子都取 I）。它对应"残差直通路径"，不经过任何 LayerNorm，所以梯度能原封不动到达任意浅层（第 9.4.1）。

10. 跟 **450 GB/s（单向）** 比。busbw 已经用 `2(N-1)/N` 修正过算法传输倍数，反映的是链路真实利用率，所以对照的是 NVLink 4.0 的单向带宽。实测 90.1 远低于 450，说明这个 8 卡环境（很可能是 PCIe 或拓扑不对）**远未跑满 NVLink**（第 15.8.2）。

11. 每层 4 次 AllReduce × 80 层 = 320 次；1 GB 数据走 NVLink（450 GB/s 单向）约 2.2 ms/次，共约 0.71 s；走 IB NDR（50 GB/s）约 20 ms/次，共约 6.4 s——**差约 9 倍**（第 15.9.1）。

12. FP16 最大有限值是 **65504**。它解释的是**溢出（overflow）**风险，不是下溢；Loss Scaling 解决的是反向小梯度**下溢为 0**。两者方向相反：scale 太大会造成溢出，所以需要动态调整（第 3.6.7）。BF16 因指数位与 FP32 相同，通常不需要 scale，但仍可能产生非有限值。

第 19 章 参考来源

19.1 AIInfraGuide 模块一（一手材料，访问日期 2026-11）

#	标题	URL
1	AI Infra 学习路线（全景三层架构 + 0.3 检验标准）	https://caomaolufei.github.io/AIInfraGuide/guides/ai-infra学习路线/
2	第 1 章：编程语言基础	https://caomaolufei.github.io/AIInfraGuide/prerequisites/模块一-前置知识/第1章-编程语言基础
3	第 2 章：数学基础	https://caomaolufei.github.io/AIInfraGuide/prerequisites/模块一-前置知识/第2章-数学基础
4	第 3 章：AI Infra 工程师学 Transformer（章首页）	https://caomaolufei.github.io/AIInfraGuide/prerequisites/模块一-前置知识/第3章-transformer架构详解
5	3.1 AI Infra 工程师为什么必须懂 Transformer	https://caomaolufei.github.io/AIInfraGuide/prerequisites/模块一-前置知识/transformer/31-ai-infra工程师为什么必须懂transformer
6	3.2 Transformer 全貌及代码实现	https://caomaolufei.github.io/AIInfraGuide/prerequisites/模块一-前置知识/transformer/32-transformer全貌及代码实现
7	3.3 Self-Attention 机制深入理解	https://caomaolufei.github.io/AIInfraGuide/prerequisites/模块一-前置知识/transformer/33-self-attention机制深入理解
8	3.4 Transformer 前馈网络 FFN 深入理解	https://caomaolufei.github.io/AIInfraGuide/prerequisites/模块一-前置知识/transformer/34-transformer前馈网络ffn深入理解
9	3.5 Transformer 位置编码深入理解	https://caomaolufei.github.io/AIInfraGuide/prerequisites/模块一-前置知识/transformer/35-transformer位置编码深入理解
10	3.6 LayerNorm 与残差连接深入理解	https://caomaolufei.github.io/AIInfraGuide/prerequisites/模块一-前置知识/transformer/36-layernorm与残差连接深入理解
11	3.7 Transformer Decoder Block 完整解析	https://caomaolufei.github.io/AIInfraGuide/prerequisites/模块一-前置知识/transformer/37-transformer-decoder-block完整解析
12	3.8 从 Transformer 到 LLM 自回归生成深入理解	https://caomaolufei.github.io/AIInfraGuide/prerequisites/模块一-前置知识/transformer/38-从transformer到llm自回归生成深入理解
13	3.9 Tokenization 与词嵌入	https://caomaolufei.github.io/AIInfraGuide/prerequisites/模块一-前置知识/transformer/39-tokenization与词嵌入
14	Transformer 架构快速入门	https://caomaolufei.github.io/AIInfraGuide/prerequisites/模块一-前置知识/transformer/transformer架构快速入门

#	标题	URL
15	第 4 章：PyTorch 框架（章首页）	https://caomaolufei.github.io/AIInfraGuide/prerequisites/模块一-前置知识/第4章-pytorch框架
16	PyTorch 框架快速入门篇	https://caomaolufei.github.io/AIInfraGuide/prerequisites/模块一-前置知识/pyroch/pytorch框架入门
17	第 5 章：GPU 硬件概论（章首页）	https://caomaolufei.github.io/AIInfraGuide/prerequisites/模块一-前置知识/第5章-gpu硬件概论
18	GPU 基础知识：从硬件架构到 AI 计算	https://caomaolufei.github.io/AIInfraGuide/prerequisites/模块一-前置知识/gpu/gpu-basics
19	NVIDIA GPU 架构演进：从 Volta 到 Blackwell	https://caomaolufei.github.io/AIInfraGuide/prerequisites/模块一-前置知识/gpu/nvidia-gpu-evolution
20	集群通信网络与 NCCL	https://caomaolufei.github.io/AIInfraGuide/prerequisites/模块一-前置知识/communication/collective-communication-primer
21	第 6 章：集合通信基础（章首页）	https://caomaolufei.github.io/AIInfraGuide/prerequisites/模块一-前置知识/第6章-集合通信基础

> 说明：第 4、5、6 章与第 3 章的"章首页"是导航页（含本章简介），正文在对应子页；第 3 章共 10 篇子页（快速入门 + 3.1～3.9），本笔记全部读取。合计 21 条 URL 覆盖模块一的 20 篇文章。

19.2 补充权威来源（用于交叉核对）

来源	用途	链接
NVIDIA CUDA C++ Programming Guide	CUDA 编程模型、算术指令吞吐	https://docs.nvidia.com/cuda/cuda-c-programming-guide/
NVIDIA A100 架构白皮书	SM 数、Tensor Core、稀疏口径	https://www.nvidia.com/en-us/data-center/a100/
NVIDIA H100 架构白皮书	132 SM、HBM3、FP8、NVLink 4.0	https://www.nvidia.com/en-us/data-center/h100/
NVIDIA H200 产品页	141 GB HBM3e、4.8 TB/s	https://www.nvidia.com/en-us/data-center/h200/
NVIDIA Blackwell 架构简介	双 die、FP4、NVLink 5.0	https://www.nvidia.com/en-us/data-center/technologies/blackwell-architecture/
NVIDIA NCCL 文档	原语语义、环境变量、算法选择	https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/

来源	用途	链接
nccl-tests	busbw 定义与测法	https://github.com/NVIDIA/nccl-tests
Vaswani et al., *Attention Is All You Need* (2017)	Self-Attention、Sinusoidal、$O(N^2)$	https://arxiv.org/abs/1706.03762
Dao et al., *FlashAttention* (2022)	tiling + online softmax、IO 复杂度	https://arxiv.org/abs/2205.14135
PyTorch 官方文档（autograd / AMP / profiler）	梯度语义、GradScaler、profiler 列含义	https://pytorch.org/docs/stable/index.html
Touvron et al., *LLaMA 2* (2023)	7B/13B/70B 配置与 GQA	https://arxiv.org/abs/2307.09288
DeepSeek-AI, *DeepSeek-V2* (2024)	MLA 与 KV 压缩率	https://arxiv.org/abs/2405.04434
Shazeer, *GLU Variants Improve Transformer* (2020)	SwiGLU 与 8/3 倍	https://arxiv.org/abs/2002.05202
Xiong et al., *On Layer Normalization in the Transformer Architecture* (2020)	Pre-Norm vs Post-Norm 方差分析	https://arxiv.org/abs/2002.04745